

If the ud-chains are represented by lists of pointers to pointers to statements (rather than by lists of pointers to statements), we can maintain ud-chains when we move statement  $s$  by simply changing the pointer to  $s$  when we move it. That is, we create for each statement  $s$  a pointer  $p_s$ , which always points to  $s$ . We put  $p_s$  on each ud-chain containing  $s$ . Then, no matter where we move  $s$ , we have only to change  $p_s$ , regardless of how many ud-chains  $s$  is on. Of course the extra level of indirection costs some compiler time and space.

If we represent ud-chains by a list of statement addresses (pointers to statements) we can still maintain ud-chains as we move statements. But then we need du-chains, as well, for efficiency. When we move  $s$ , we may go down its du-chain, changing the ud-chain of all uses that refer to  $s$ .

The dominator information is changed slightly by code motion. The preheader is now the immediate dominator of the header, and the immediate dominator of the preheader is the node that formerly was the immediate dominator of the header. That is, the preheader is inserted into the dominator tree as the parent of the header.

### Elimination of Induction Variables

A variable  $x$  is called an *induction variable* of a loop  $L$  if every time the variable  $x$  changes values, it is incremented or decremented by some constant. Often, an induction variable is incremented by the same constant each time around the loop, as  $i$  in a loop headed by **for**  $i := 1$  **to** 10. However, our methods deal with variables that are incremented or decremented zero, one, two, or more times as we go around a loop. The number of changes to an induction variable may even differ at different iterations.

A common situation is one in which an induction variable, say  $i$ , indexes an array, and some other induction variable, say  $t$ , whose value is a linear function of  $i$ , is the actual offset used to access the array. Often, the only use made of  $i$  is in the test for loop termination. We can then get rid of  $i$  by replacing its test by one on  $t$ .

The algorithms that follow deal with a restricted class of induction variables to simplify the presentation. Some extensions of the algorithms can be done by adding more cases, but others require theorems to be proved about expressions involving the usual arithmetic operators.

We shall look for *basic induction variables*, which are those variables  $i$  whose only assignments within loop  $L$  are of the form  $i := i \pm c$ , where  $c$  is a constant.<sup>7</sup> We then look for additional induction variables  $j$  that are defined only once within  $L$ , and whose value is a linear function of some basic induction variable  $i$  where  $j$  is defined.

<sup>7</sup> In our discussion of induction variables, "+" stands only for the addition operator, not a generic operator, and likewise for the other standard arithmetic operators.

**Algorithm 10.9.** Detection of induction variables.

**Input.** A loop  $L$  with reaching definition information and loop-invariant computation information (from Algorithm 10.7).

**Output.** A set of induction variables. Associated with each induction variable  $j$  is a triple  $(i, c, d)$ , where  $i$  is a basic induction variable, and  $c$  and  $d$  are constants such that the value of  $j$  is given by  $c*i+d$  at the point where  $j$  is defined. We say that  $j$  belongs to the *family* of  $i$ . The basic induction variable  $i$  belongs to its own family.

**Method.**

1. Find all basic induction variables by scanning the statements of  $L$ . We use the loop-invariant computation information here. Associated with each basic induction variable  $i$  is the triple  $(i, 1, 0)$ .
2. Search for variables  $k$  with a single assignment to  $k$  within  $L$  having one of the following forms:

$$k := j * b, \quad k := b * j, \quad k := j / b, \quad k := j \pm b, \quad k := b \pm j$$

where  $b$  is a constant, and  $j$  is an induction variable, basic or otherwise.

If  $j$  is basic, then  $k$  is in the family of  $j$ . The triple for  $k$  depends on the instruction defining it. For example, if  $k$  is defined by  $k := j * b$ , then the triple for  $k$  is  $(j, b, 0)$ . The triples for the remaining cases can be determined similarly.

If  $j$  is not basic, let  $j$  be in the family of  $i$ . Then our additional requirements are that

- (a) there is no assignment to  $i$  between the lone point of assignment to  $j$  in  $L$  and the assignment to  $k$ , and
- (b) no definition of  $j$  outside  $L$  reaches  $k$ .

The usual case will be where the definitions of  $k$  and  $j$  are in temporaries in the same block, in which case it is easy to check. In general, reaching definitions information will provide the check we need if we analyze the flow graph of the loop  $L$  to determine those blocks (and therefore those definitions) on paths between the assignment to  $j$  and the assignment to  $k$ .

We compute the triple for  $k$  from the triple  $(i, c, d)$  for  $j$  and the instruction defining  $k$ . For example, the definition  $k := b * j$  leads to  $(i, b * c, b * d)$  for  $k$ . Note that the multiplications in  $b * c$  and  $b * d$  can be done as the analysis proceeds because  $b$ ,  $c$ , and  $d$  are constants.  $\square$

Once families of induction variables have been found, we modify the instructions computing an induction variable to use additions or subtractions rather than multiplications. The replacement of a more expensive instruction by a cheaper one is called *strength reduction*.

**Example 10.23.** The loop consisting of block  $B_2$  in Fig. 10.39(a) has basic induction variable  $i$ , because the lone assignment to  $i$  in the loop increments its value by 1. The family of  $i$  contains  $t_2$ , because there is a single assignment to  $t_2$ , with right hand side  $4*i$ . Thus the triple for  $t_2$  is  $(i, 4, 0)$ . Similarly,  $j$  is the only basic induction variable in the loop consisting of  $B_3$ , and  $t_4$ , with triple  $(j, 4, 0)$ , is in the family of  $j$ .

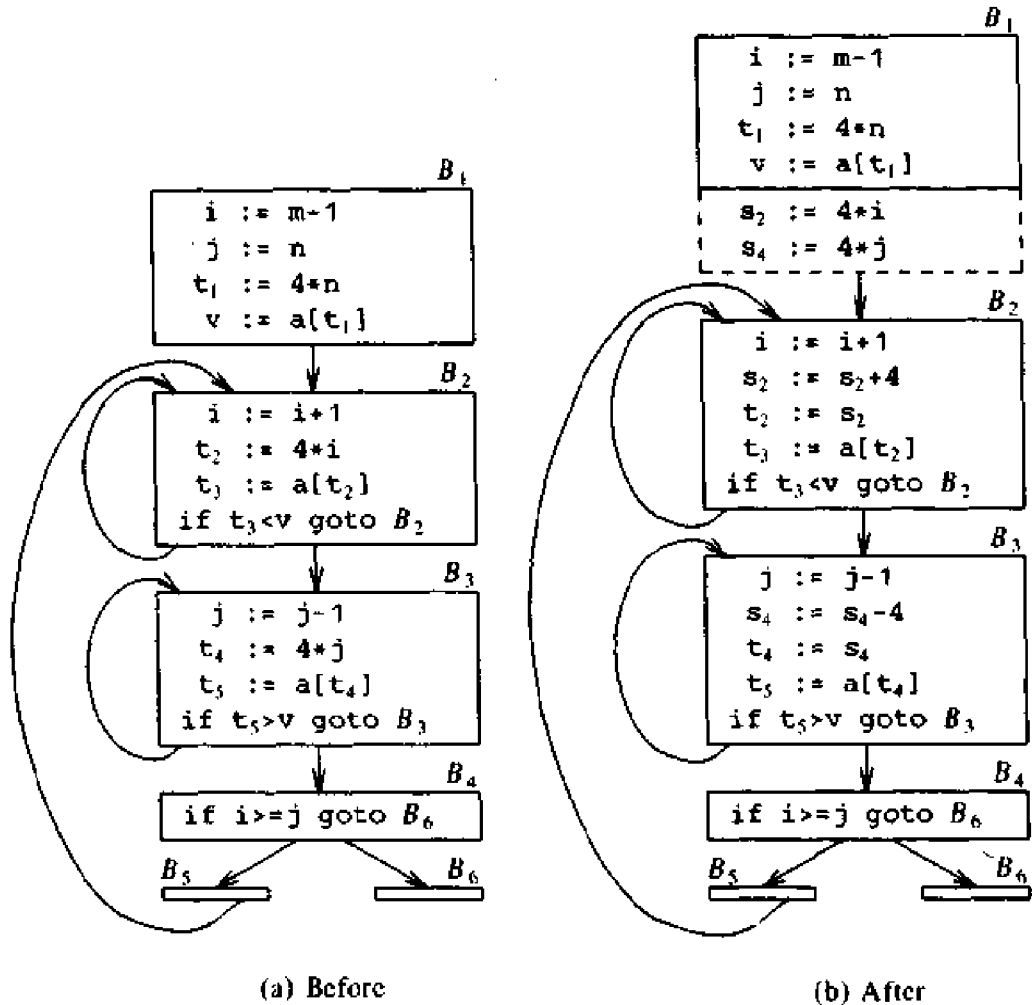


Fig. 10.39. Strength reduction.

We can also look for induction variables in the outer loop with header  $B_1$  and blocks  $B_2$ ,  $B_3$ ,  $B_4$ ,  $B_5$ . Both  $i$  and  $j$  are basic induction variables in this larger loop. Again  $t_2$  and  $t_4$  are induction variables with triples  $(i, 4, 0)$  and  $(j, 4, 0)$ , respectively.

The flow graph in Fig. 10.39(b) is obtained from that in Fig. 10.39(a) by applying the next algorithm. We shall discuss this transformation below.  $\square$

**Algorithm 10.10.** Strength reduction applied to induction variables.

*Input.* A loop  $L$  with reaching definition information and families of induction variables computed using Algorithm 10.9.

*Output.* A revised loop.

*Method.* Consider each basic induction variable  $i$  in turn. For every induction variable  $j$  in the family of  $i$  with triple  $(i, c, d)$ :

1. Create a new variable  $s$  (but if two variables  $j_1$  and  $j_2$  have the same triples, just create one new variable for both).
2. Replace the assignment to  $j$  by  $j := s$ .
3. Immediately after each assignment  $i := i + n$  in  $L$ , where  $n$  is a constant, append:

$$s := s + c * n$$

where the expression  $c * n$  is evaluated to a constant because  $c$  and  $n$  are constants. Place  $s$  in the family of  $i$ , with triple  $(i, c, d)$ .

4. It remains to ensure that  $s$  is initialized to  $c * i + d$  on entry to the loop. The initialization may be placed at the end of the preheader. The initialization consists of

$$\begin{aligned} s &:= c * i && /* \text{ just } s := i \text{ if } c \text{ is } 1 */ \\ s &:= s + d && /* \text{ omit if } d \text{ is } 0 */ \end{aligned}$$

Note that  $s$  is an induction variable in the family of  $i$ . □

**Example 10.24.** Suppose we consider the loops in Fig. 10.39(a) inside-out. Since the treatment of inner loops containing  $B_2$  and  $B_3$  is very similar, we shall talk of the loop around  $B_3$  only. In Example 10.23 we observed that the basic induction variable in the loop around  $B_3$  is  $j$ , and the other induction variable is  $t_4$  with triple  $(j, 4, 0)$ . In step (1) of Algorithm 10.10, a new variable  $s_4$  is constructed. In step (2), the assignment  $t_4 := 4 * j$  is replaced by  $t_4 := s_4$ . Step (4) inserts the assignment  $s_4 := s_4 - 4$  after the assignment  $j := j - 1$ , where the  $-4$  is obtained by multiplying the  $-1$  in the assignment to  $j$  and the  $4$  in the triple  $(j, 4, 0)$  for  $t_4$ .

Since  $B_1$  serves as a preheader for the loop, we may place the initialization of  $s_4$  at the end of the block  $B_1$  containing the definition of  $j$ . The added instructions are shown in the dashed extension to block  $B_1$ .

When the outer loop is considered, the flow graph looks as shown in Fig. 10.39(b). There are four variables,  $i$ ,  $s_2$ ,  $j$ , and  $s_4$ , that could be considered induction variables. However, step (3) of Algorithm 10.10 places the newly created variables in the families of  $i$  and  $j$ , respectively, to facilitate the elimination of  $i$  and  $j$ , using the next algorithm. □

After strength reduction we find that the only use of some induction variables is in tests; we can replace a test of such an induction variable by that of

another. For example, if  $i$  and  $t$  are induction variables such that the value of  $t$  is always four times the value of  $i$ , then the test  $i \geq j$  is equivalent to  $t \geq 4*j$ . After this replacement, it may be possible to eliminate  $i$ . Note however that if  $t = -4*i$ , then we need to change the relational operator as well, because  $i \geq j$  is equivalent to  $t \leq -4*j$ . In the following algorithm we consider the case where the multiplicative constant is positive, leaving the generalization to negative constants as an exercise.

**Algorithm 10.11.** Elimination of induction variables.

*Input.* A loop  $L$  with reaching definition information, loop-invariant computation information (from Algorithm 10.7) and live variable information.

*Output.* A revised loop.

*Method.*

1. Consider each basic induction variable  $i$  whose only uses are to compute other induction variables in its family and in conditional branches. Take some  $j$  in  $i$ 's family, preferably one such that  $c$  and  $d$  in its triple  $(i, c, d)$  are as simple as possible (i.e., we prefer  $c = 1$  and  $d = 0$ ), and modify each test that  $i$  appears in to use  $j$  instead. We assume in the following that  $c$  is positive. A test of the form **if**  $i$  **relop**  $x$  **goto**  $B$ , where  $x$  is not an induction variable, is replaced by

```

r := c*x          /* r := x if c is 1 */
r := r+d          /* omit if d is 0 */
if  $j$  relop  $r$  goto  $B$ 

```

where  $r$  is a new temporary. The case **if**  $x$  **relop**  $i$  **goto**  $B$  is handled analogously. If there are two induction variables  $i_1$  and  $i_2$  in the test **if**  $i_1$  **relop**  $i_2$  **goto**  $B$ , then we check if both  $i_1$  and  $i_2$  can be replaced. The easy case is when we have  $j_1$  with triple  $(i_1, c_1, d_1)$  and  $j_2$  with triple  $(i_2, c_2, d_2)$ , and  $c_1 = c_2$  and  $d_1 = d_2$ . Then,  $i_1$  **relop**  $i_2$  is equivalent to  $j_1$  **relop**  $j_2$ . In more complex cases, replacement of the test may not be worthwhile, because we may need to introduce two multiplicative steps and one addition, while as few as two steps may be saved by eliminating  $i_1$  and  $i_2$ .

Finally, delete all assignments to the eliminated induction variables from the loop  $L$ , as they will now be useless.

2. Now, consider each induction variable  $j$  for which a statement  $j := s$  was introduced by Algorithm 10.10. First check that there can be no assignment to  $s$  between the introduced statement  $j := s$  and any use of  $j$ . In the usual situation,  $j$  is used in the block in which it is defined, simplifying this check; otherwise, reaching definitions information, plus some graph analysis is needed to implement the check. Then replace all uses of  $j$  by uses of  $s$  and delete statement  $j := s$ . □

**Example 10.25.** Consider the flow graph in Fig. 10.39(b). The inner loop around  $B_2$  contains two induction variables  $i$  and  $s_2$ , but neither can be eliminated because  $s_2$  is used to index into the array  $a$  and  $i$  is used in a test outside the loop. Similarly, the loop around  $B_3$  contains induction variables  $j$  and  $s_4$ , but neither can be eliminated.

Let us apply Algorithm 10.11 to the outer loop. When the new variables  $s_2$  and  $s_4$  were created by Algorithm 10.10, as discussed in Example 10.24,  $s_2$  was placed in the family of  $i$  and  $s_4$  was placed in the family of  $j$ . Consider the family of  $i$ . The only use of  $i$  is in the test for loop termination in block  $B_4$ , so  $i$  is a candidate for elimination in step (1) of Algorithm 10.11. The test in block  $B_4$  involves the two induction variables  $i$  and  $j$ . Fortunately, the families of  $i$  and  $j$  contain  $s_2$  and  $s_4$  with the same constants in their triples, because the triples are  $(i, 4, 0)$  and  $(j, 4, 0)$ , respectively. The test  $i \geq j$  can therefore be replaced by  $s_2 \geq s_4$ , allowing both  $i$  and  $j$  to be eliminated.

Step (2) of the Algorithm 10.11 applies copy propagation to the newly created variables, replacing  $t_2$  and  $t_4$  by  $s_2$  and  $s_4$ , respectively.  $\square$

### Induction Variables with Loop-Invariant Expressions

In Algorithms 10.9 and 10.10 we can allow loop-invariant expressions instead of constants. However, the triple  $(i, c, d)$  for an induction variable  $j$  may then contain loop-invariant expressions rather than constants. The evaluation of these expressions should be done outside the loop  $L$ , in the preheader. Moreover, since the intermediate code requires there to be at most one operator per instruction, we must be prepared to generate intermediate code statements for the evaluation of the expressions. The replacement of tests in Algorithm 10.11 requires the sign of the multiplicative constant  $c$  to be known. For this reason it may be reasonable to restrict attention to cases in which  $c$  is a known constant.

## 10.8 DEALING WITH ALIASES

If two or more expressions denote the same memory address, we say that the expressions are *aliases* of one another. In this section we shall consider data-flow analysis in the presence of pointers and procedures, both of which introduce aliases.

The presence of pointers makes data-flow analysis more complex, since they cause uncertainty regarding what is defined and used. The only safe assumption if we know nothing about where pointer  $p$  can point to is to assume that an indirect assignment through a pointer can potentially change (i.e., define) any variable. We must also assume that any use of the data pointed to by a pointer, e.g.,  $x := *p$ , can potentially use any variable. These assumptions result in more live variables and reaching definitions than is realistic and fewer available expressions than is realistic. Fortunately, we can use data-

flow analysis to pin down what a pointer might point to, thus allowing us to get more accurate information from our other data-flow analyses.

As for assignments with pointer variables, when we come to a procedure call, we may not have to make our worst-case assumption — that everything can be changed — provided, we can compute the set of variables that the procedure might change. As with all code optimizations we may still make errors on the conservative side. That is, the sets of variables whose values “may be” changed or used could properly include the variables that are actually changed or used in some execution of the program. We shall, as usual, simply try to come fairly close to the true sets of changed and used variables without working unduly hard or making an error that alters what the program does.

### **A Simple Pointer Language**

For specificity, let us consider a language in which there are elementary data types (e.g., integers and reals) requiring one word each, and arrays of these types. Let there also be pointers to these elements and to arrays, but not to other pointers. We shall be content to know that a pointer *p* is pointing somewhere in array *a*, without concerning ourselves with what particular element of *a* is being pointed to. This grouping together of all elements of an array, as far as pointer targets are concerned, is reasonable. Typically, pointers will be used as cursors to run through an entire array, so a more detailed data-flow analysis, if we could accomplish it at all, would often tell us that at a particular point in the program, *p* might be pointing to any one of the elements of *a* anyway.

We must also make certain assumptions about which arithmetic operations on pointers are semantically meaningful. First, if pointer *p* points to a primitive (one word) data element, then any arithmetic operation on *p* produces a value that may be an integer, but not a pointer. If *p* points to an array, then addition or subtraction of an integer leaves *p* pointing somewhere in the same array, while other arithmetic operations on pointers produce a value that is not a pointer. While not all languages prohibit, say, moving a pointer from one array *a* to another array *b* by adding to the pointer, such action would depend on the particular implementation to make sure that array *b* followed *a* in storage. It is our point of view that an optimizing compiler should adhere only to the language definition in deciding what optimizations to perform. Each compiler implementer, however, must make a judgment on what specific optimizations the compiler should be allowed to do.

### **The Effects of Pointer Assignments**

Under these assumptions, the only variables that could possibly be used as pointers are those declared to be pointers and temporaries that receive a value that is a pointer plus or minus a constant. We shall refer to all these variables as pointers. Our rules for determining what a pointer *p* can point to are as follows.

1. If there is an assignment statement  $s: p := \&a$ , then immediately after  $s$ ,  $p$  points only to  $a$ . If  $a$  is an array, then  $p$  can point only to  $a$  after any assignment to  $p$  of the form  $p := \&a \pm c$ , where  $c$  is a constant.<sup>\*</sup> As usual,  $\&a$  is deemed to refer to the location of the first element of array  $a$ .
2. If there is an assignment statement  $s: p := q \pm c$ , where  $c$  is an integer other than zero, and  $p$  and  $q$  are pointers, then immediately after  $s$ ,  $p$  can point to any array that  $q$  could point to before  $s$ , but to nothing else.
3. If there is an assignment  $s: p := q$ , then immediately after  $s$ ,  $p$  can point to whatever  $q$  could point to before  $s$ .
4. After any other assignment to  $p$ , there is no object that  $p$  could point to; such an assignment is probably (depending on the semantics of the language) meaningless.
5. After any assignment to a variable other than  $p$ ,  $p$  points to whatever it did before the assignment. Note that this rule assumes that no pointer can point to a pointer. Relaxing this assumption does not make matters particularly more difficult, and we leave the generalization to the reader.

We shall define  $in[B]$ , for block  $B$ , to be the function that gives for each pointer  $p$  the set of variables to which  $p$  could point at the beginning of  $B$ . Formally,  $in[B]$  is a set of pairs of the form  $(p, a)$ , where  $p$  is a pointer and  $a$  a variable, meaning that  $p$  might point to  $a$ . In practice,  $in[B]$  might be represented as a list for each pointer, the list for  $p$  giving the set of  $a$ 's such that  $(p, a)$  is in  $in[B]$ . We define  $out[B]$  similarly for the end of  $B$ .

We specify a transfer function  $trans_B$  that defines the effect of block  $B$ . That is,  $trans_B$  is a function taking as argument a set of pairs  $S$ , each pair of the form  $(p, a)$  for  $p$  a pointer and  $a$  a nonpointer variable, and producing another set  $T$ . Presumably, the set to which  $trans_B$  is applied will be  $in[B]$  and the result of the application will be  $out[B]$ . We need only tell how to compute  $trans$  for single statements;  $trans_B$  will then be the composition of  $trans_s$  for each statement  $s$  of block  $B$ . The rules for computing  $trans$  are as follows.

1. If  $s$  is  $p := \&a$  or  $p := \&a \pm c$  in the case  $a$  is an array, then

$$trans_s(S) = (S - \{(p, b) \mid \text{any variable } b\}) \cup \{(p, a)\}$$

2. If  $s$  is  $p := q \pm c$  for pointer  $q$  and nonzero integer  $c$ , then

$$trans_s(S) = (S - \{(p, b) \mid \text{any variable } b\}) \\ \cup \{(p, b) \mid (q, b) \text{ is in } S \text{ and } b \text{ is an array variable}\}$$

Note that this rule makes sense even if  $p = q$ .

3. If  $s$  is  $p := q$ , then

<sup>\*</sup> In this section,  $+$  stands for itself rather than a generic operator.



$$\begin{aligned} trans_s(S) = & (S - \{(p, b) \mid \text{any variable } b\}) \\ & \cup \{(p, b) \mid (q, b) \text{ is in } S\} \end{aligned}$$

4. If  $s$  assigns to pointer  $p$  any other expression, then

$$trans_s(S) = S - \{(p, b) \mid \text{any variable } b\}$$

5. If  $s$  is not an assignment to a pointer, then  $trans_s(S) = S$ .

We may now write the equations relating *in*, *out*, and *trans* as follows.

$$\begin{aligned} out[B] &= trans_B(in[B]) \\ in[B] &= \bigcup_{\substack{P \text{ a pred-} \\ \text{ecessor of } B}} out[P] \end{aligned} \quad (10.13)$$

where if  $B$  consists of statements  $s_1, s_2, \dots, s_k$ , then

$$trans_B(S) = trans_{s_k}(trans_{s_{k-1}}(\dots(trans_{s_2}(trans_{s_1}(S)))\dots)).$$

Equations 10.13 can be solved essentially like reaching definitions in Algorithm 10.2. We shall therefore not go into the details of the algorithm, but content ourselves with an example.

**Example 10.26.** Consider the flow graph of Fig. 10.40. We suppose  $a$  is an array and  $c$  is an integer;  $p$  and  $q$  are pointers. Initially, we set  $in[B_1]$  to  $\emptyset$ . Then,  $trans_{B_1}$  has the effect of removing any pairs with first component  $q$ , then adding the pair  $(q, c)$ . That is,  $q$  is asserted to point to  $c$ . Thus

$$out[B_1] = trans_{B_1}(\emptyset) = \{(q, c)\}$$

Then,  $in[B_2] = out[B_1]$ . The effect of  $p := \&c$  is to replace all pairs with first component  $p$  by the pair  $(p, c)$ . The effect of  $q := \&(a[2])$  is to replace pairs with first component  $q$  by  $(q, a)$ . Note that  $q := \&(a[2])$  is actually an assignment of the form  $q := \&a + c$  for a constant  $c$ . We may now compute

$$out[B_2] = trans_{B_2}(\{(q, c)\}) = \{(p, c), (q, a)\}$$

Similarly,  $in[B_3] = \{(q, c)\}$  and  $out[B_3] = \{(p, a), (q, c)\}$ .

Next, we find  $in[B_4] = out[B_2] \cup out[B_3] \cup out[B_5]$ . Presumably,  $out[B_5]$  was initialized to  $\emptyset$  and has not been changed on this pass yet. However,  $out[B_2] = \{(p, c), (q, a)\}$  and  $out[B_3] = \{(p, a), (q, c)\}$ , so

$$in[B_4] = \{(p, a), (p, c), (q, a), (q, c)\}$$

The effect of  $p := p + 1$  in  $B_4$  is to discard the possibility that  $p$  does not point to an array. That is,

$$out[B_4] = trans_{B_4}(in[B_4]) = \{(p, a), (q, a), (q, c)\}$$

Note that whenever  $B_2$  is executed, making  $p$  point to  $c$ , a semantically meaningless action takes place if  $p$  is used indirectly after  $p := p + 1$  in  $B_4$ . Thus, this flow graph is not "realistic," but it does illustrate the inferences about pointers that we can make.

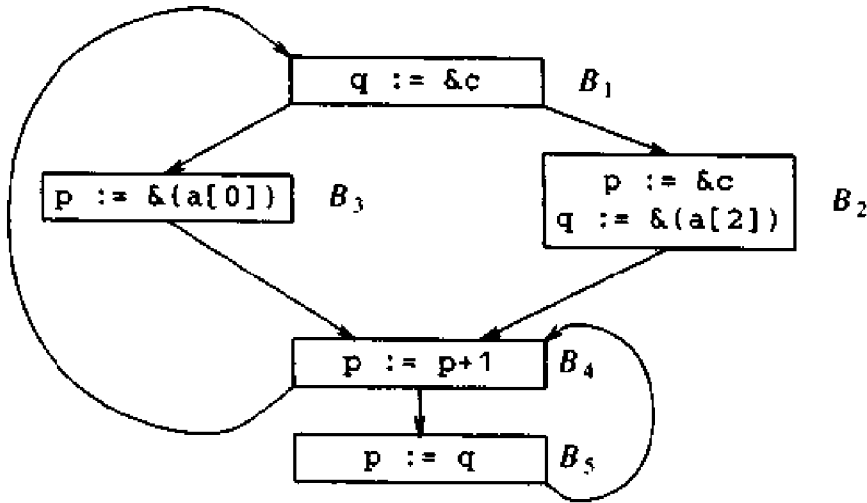


Fig. 10.40. Flow graph with pointer operations shown.

Continuing,  $in[B_5] = out[B_4]$ , and  $trans_{B_5}$  copies the targets of  $q$  and gives them to  $p$  as well. Since  $q$  can point to  $a$  or  $c$  in  $in[B_5]$ ,

$$out[B_5] = \{(p, a), (p, c), (q, a), (q, c)\}$$

On the next pass we find  $in[B_1] = out[B_4]$ , so  $out[B_1] = \{(p, a), (q, c)\}$ . This value is also the new  $in[B_2]$  and  $in[B_3]$ , but these new values do not change  $out[B_2]$  or  $out[B_3]$ , nor is  $in[B_4]$  changed. We have thus converged to the desired answer.  $\square$

### Making Use of Pointer Information

Suppose  $in[B]$  is the set of variables pointed to by each pointer at the beginning of block  $B$  and that we have a reference to pointer  $p$  inside block  $B$ . Starting with  $in[B]$ , apply  $trans_s$  for each statement  $s$  of block  $B$  that precedes the reference to  $p$ . This computation tells us what  $p$  could point to at the particular statement where that information is important.

Suppose now that we have determined what each pointer could point to when that pointer is used in an indirect reference, either on the left or right of the assignment symbol. How can we use this information to get more accurate solutions to the usual data-flow problems? In each case we must consider in which direction errors are conservative, and we must utilize pointer information in such a way that only conservative errors are made. To see how this choice is made, let us consider two examples: reaching definitions and live variable analysis.

To calculate reaching definitions we can use Algorithm 10.2, but we need to know the values of  $kill$  and  $gen$  for a block. The latter quantities are computed as usual for statements that are not indirect assignments through pointers. An indirect assignment  $*p := a$  is deemed to generate a definition of

every variable  $b$  such that  $p$  could point to  $b$ . This assumption is conservative, because as discussed in Section 10.5, it is generally conservative to assume definitions reach a point while in reality they do not.

When we compute *kill*, we shall assume that  $*p:=a$  kills definitions of  $b$  only if  $b$  is not an array and is the only variable  $p$  could possibly point to. If  $p$  could point to two or more variables, then we do not assume definitions of either are killed. Again, we are being conservative because we permit definitions of  $b$  to pass through  $*p:=a$ , and thus reach wherever they can, unless we can prove that  $*p:=a$  redefined  $b$ . In other words, when there is doubt, we assume that a definition reaches.

For live variables we may use Algorithm 10.4, but we must reconsider how *def* and *use* are to be defined for statements of the form  $*p:=a$  and  $a:=*p$ . The statement  $*p:=a$  uses only  $a$  and  $p$ . We say it defines  $b$  only if  $b$  is the unique variable that  $p$  might point to. This assumption allows uses of  $b$  to pass through unless they are surely blocked by the assignment  $*p:=a$ . Thus we can never claim that  $b$  is dead at a point when it is in fact live. The statement  $a:=*p$  surely represents a definition of  $a$ . It also represents a use of  $p$  and a use of any variable that  $p$  could point to. By maximizing possible uses, we again maximize our estimate of live variables. By maximizing live variables, we are normally being conservative. For example, we might generate code to store a dead variable, but we shall never fail to store one that was live.

### Interprocedural Data Flow Analysis

Until now, we have spoken of “programs” that are single procedures and therefore single flow graphs. We shall now see how to gather information from many interacting procedures. The basic idea is to determine how each procedure influences the *gen*, *kill*, *use*, or *def* information of the others, then to compute our data-flow information for each procedure independently as before.

During data-flow analysis we shall have to deal with aliases set up by parameters in procedure calls. As it is not possible for two global variables to denote the same memory address, at least one of a pair of aliases must be a formal parameter. Since formals may be passed to procedures, it is possible for two formals to be aliases.

**Example 10.27.** Suppose we have a procedure  $p$ , with two formal parameters  $x$  and  $y$  passed by reference. In Fig. 10.41, we see a situation in which  $b+x$  is computed in  $B_1$  and  $B_3$ . Suppose that the only paths from blocks  $B_1$  to  $B_3$  go through  $B_2$ , and there are no assignments to  $b$  or  $x$  along any such path. Then, is  $b+x$  available at  $B_3$ ? The answer depends on whether  $x$  and  $y$  could denote the same memory address. For example, there could be a call  $p(z, z)$ , or perhaps a call of  $p(u, v)$ , where  $u$  and  $v$  are formal parameters of another procedure  $q(u, v)$ , and a call of  $q(z, z)$  is possible.

Similarly, it is possible for  $x$  and  $y$  to be aliases if  $x$  is a formal parameter,

say of  $p(x, w)$ , and  $y$  is a variable with a scope accessible to some procedure  $q$  that calls  $p$ , say with call  $p(y, t)$ . Even more complicated situations could make  $x$  and  $y$  aliases of one another, and we shall shortly develop some general rules for determining all such pairs of aliases.  $\square$

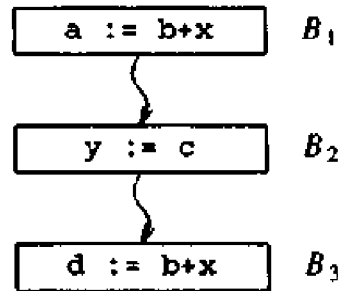


Fig. 10.41. Illustration of aliasing problems.

It will turn out in some situations that it is conservative not to regard variables as aliases of one another. For example, in reaching definitions, if we wish to assert that a definition of  $a$  is killed by a definition of  $b$ , we had better be sure that  $a$  and  $b$  are definitely aliases whenever the definition of  $b$  is executed. Other times it is conservative to regard variables as aliases of one another whenever there is doubt. Example 10.27 is one such case. If the available expression  $b+x$  is not to be killed by a definition of  $y$ , we had better be sure neither  $b$  nor  $x$  could be an alias of  $y$ .

### A Model of Code with Procedure Calls

To illustrate how aliasing might be dealt with, let us consider a language that permits recursive procedures, any of which may refer to both local and global variables. The data available to a procedure consists of the globals and its own locals only; that is, there is no block structure to the language. Parameters are passed by reference. We require that all procedures have a flow graph with single *entry* (the initial node) and a single *return* node which causes control to pass back to the calling routine. We suppose for convenience that every node lies on some path from the entry to the return.

Now suppose we are in a procedure  $p$  and we come upon a call to procedure  $q(u, v)$ . If we are interested in computing reaching definitions, available expressions or any of a number of other data-flow analyses, we must know whether  $q(u, v)$  might change the value of some variable. Note that we say "might" change, rather than "will" change. As with all data-flow problems, it is impossible to know for certain whether or not the value of a variable is changed. We can only find a set that includes all variables whose values do change and perhaps some that don't. By being careful, we can cut

down on the latter class of variables, obtaining a good approximation to the truth and erring only on the conservative side.

The only variables whose values the call of  $q(u, v)$  could define are the globals and the variables  $u$  and  $v$ , which might be local to  $p$ . Definitions of locals of  $q$  are of no consequence after the call returns. Even if  $p = q$ , it will be other copies of  $q$ 's locals that change, and these copies disappear after the return. It is easy to determine which globals are explicitly defined by  $q$ ; just see which have definitions in  $q$ , or are defined in a procedure call made by  $q$ . In addition,  $u$  and/or  $v$ , which may be global, change if  $q$  has a definition of its first and/or second formal parameter, respectively, or if these formal parameters are passed as actual parameters by  $q$  to another procedure that defines them. However, not every variable changed by a call to  $q$  need be defined explicitly by  $q$  or one of the procedures it calls, because variables may have aliases.

### Computing Aliases

Before we can answer the question of what variables might change in a given procedure, we must develop an algorithm for finding aliases. The approach we shall use here is a simple one. We compute a relation  $\equiv$  on variables that formalizes the notion "can be an alias of." In so doing we do not distinguish among occurrences of a variable in different calls to the same procedure, although we distinguish variables local to different procedures but having the same identifier.

To make things simpler, we do not try to differentiate the sets of aliases at different points in the program. Rather, if two variables could ever be aliases of one another we assume they always could be. Finally, we shall make the conservative assumption that  $\equiv$  is transitive, so variables are grouped into equivalence classes, and two variables could be aliases of one another if and only if they are in the same class.

**Algorithm 10.12.** Simple alias computation.

*Input.* A collection of procedures and global variables.

*Output.* An equivalence relation  $\equiv$  with the property that whenever there is a position in the program where  $x$  and  $y$  are aliases of one another,  $x \equiv y$ ; the converse need not be true.

*Method.*

1. Rename variables, if necessary, so that no two procedures use the same formal parameter or local variable identifier, nor do a local, formal, or global share an identifier.
2. If there is a procedure  $p(x_1, x_2, \dots, x_n)$  and an invocation  $p(y_1, y_2, \dots, y_n)$  of that procedure, set  $x_i \equiv y_i$  for all  $i$ . That is, each formal parameter can be an alias of any of its corresponding actual parameters.

3. Take the reflexive and transitive closure of the actual-formal correspondences by adding

- a)  $x = y$  whenever  $y = x$ .
- b)  $x = z$  whenever  $x = y$  and  $y = z$  for some  $y$ . □

**Example 10.28.** Consider the sketch of the three procedures shown in Fig. 10.42, where parameters are assumed passed by reference. There are two globals,  $g$  and  $h$ , and two local variables,  $i$  for the procedure `main` and  $k$  for the procedure `two`. Procedure `one` has formals  $w$  and  $x$ , procedure `two` has formals  $y$  and  $z$ , and `main` has no formal parameters. Thus, no renaming of variables is necessary. We first compute the aliasing due to actual-formal correspondences.

The call of `one` by `main` makes  $h = w$  and  $i = x$ . The first call of `two` by `one` makes  $w = y$  and  $w = z$ . The second call makes  $g = y$  and  $x = z$ .

The call of `one` by `two` makes  $k = w$  and  $y = x$ . When we take the transitive closure of the alias relationships represented by  $=$ , then we find in this example that all variables are possible aliases of one another. □

```
global g, h;
  procedure main( );
    local i;
    g := ... ;
    one(h, i)
  end
  procedure one(w, x);
    x := ... ;
    two(w, w);
    two(g, x)
  end;
  procedure two(y, z);
    local k;
    h := ... ;
    one(k, y)
  end
```

Fig. 10.42. Sample procedures.

The aliasing computation of Algorithm 10.12 does not often result in such extensive aliasing as we found in Example 10.28. Intuitively, we would not often expect two different variables with dissimilar types to be aliases. Moreover, the programmer undoubtedly has conceptual types for his variables. For example, if the first formal parameter of a procedure  $p$  represents a velocity, it can be expected that the first argument in any call to  $p$  will also be thought

of by the programmer as a velocity. Thus we intuitively expect most programs to produce small groups of possible aliases.

### Data-Flow Analysis in the Presence of Procedure Calls

Let us consider, as an example, how available expressions can be calculated in the presence of procedure calls, where parameters are passed by reference. As in Section 10.6, we must determine when a variable could be defined, thus killing an expression, and we must determine when expressions are generated (evaluated).

We may define, for each procedure  $p$ , a set  $change[p]$ , whose value is to be the set of global variables and formal parameters of  $p$  that might be changed during an execution of  $p$ . At this point, we do not count a variable as changed if a member of its equivalence class of aliases is changed.

Let  $def[p]$  be the set of formal parameters and globals having explicit definitions within  $p$  (not including those defined within procedures called by  $p$ ). To write the equations for  $change[p]$ , we have only to relate the globals and formals of  $p$  that are used as actual parameters in calls made by  $p$  to the corresponding formal parameters of the called procedures. We may write:

$$change[p] = def[p] \cup A \cup G \quad (10.14)$$

where

1.  $A = \{a \mid a \text{ is a global variable or formal parameter of } p \text{ such that, for some procedure } q \text{ and integer } i, p \text{ calls } q \text{ with } a \text{ as the } i\text{th actual parameter and the } i\text{th formal parameter of } q \text{ is in } change[q]\}$
2.  $G = \{g \mid g \text{ is a global in } change[q] \text{ and } p \text{ calls } q\}$ .

It should come as no surprise that Equation (10.14) may be solved for a set of procedures by an iterative technique. Although the solution is not unique, we only need the smallest one. We may converge to that solution by starting with a too-small approximation and iterating. The obvious too-small approximation with which to start is  $change[p] = def[p]$ . The details of the iteration are left as an exercise for the reader.

It is worth considering the order in which procedures should be visited in the above iteration. For example, if the procedures are not mutually recursive, then we can first visit procedures that do not call any other (there must be at least one). For these procedures,  $change = def$ . Next, we can compute  $change$  for those procedures that call only procedures which call nothing. We can apply (10.14) for this next group of procedures directly, since  $change[q]$  will be known for any  $q$  in (10.14).

This idea can be made more precise as follows. We draw a *calling graph*, whose nodes are procedures, with an edge from  $p$  to  $q$  if  $p$  calls  $q$ .<sup>9</sup> A

<sup>9</sup> Here we assume no procedure-valued variables. These complicate the construction of the calling graph, as we must determine the possible actuals corresponding to procedure-valued formals at the same time we construct the edges of the calling graph.

collection of procedures that are not mutually recursive will have an acyclic calling graph. In this case we can visit each node once.

We now give an algorithm for computing *change*.

**Algorithm 10.13.** Interprocedural analysis of changed variables.

*Input.* A collection of procedures  $p_1, p_2, \dots, p_n$ . If the calling graph is acyclic, we assume  $p_i$  calls  $p_j$  only if  $j < i$ . Otherwise, we make no assumption about which procedures call which.

*Output.* For each procedure  $p$ , we produce  $change[p]$ , the set of global variables and formal parameters of  $p$  that may be changed explicitly by  $p$  with no aliasing.

*Method.*

1. Compute  $def[p]$  for each procedure  $p$  by inspection.
2. Execute the program of Fig. 10.43 to compute *change*. □

```

(1) for each procedure  $p$  do  $change[p] := def[p]$ ; /* initialize */
(2) while changes to any  $change[p]$  occur do
(3)   for  $i := 1$  to  $n$  do
(4)     for each procedure  $q$  called by  $p_i$  do begin
(5)       add any global variables in  $change[q]$  to  $change[p_i]$ ;
(6)       for each formal parameter  $x$  (the  $j$ th) of  $q$  do
(7)         if  $x$  is in  $change[q]$  then
(8)           for each call of  $q$  by  $p_i$  do
(9)             if  $a$ , the  $j$ th actual parameter of the call,
                is a global or formal parameter of  $p_i$ 
(10)            then add  $a$  to  $change[p_i]$ 
      end

```

**Fig. 10.43.** Iterative algorithm to compute *change*.

**Example 10.29.** Let us consider Fig. 10.42 again. By inspection,  $def[main] = \{g\}$ ,  $def[one] = \{x\}$  and  $def[two] = \{h\}$ . These are the initial values of *change*. The calling graph of the procedures is shown in Fig. 10.44. We shall take two, one, main as the order in which we visit the procedures.

Consider  $p_i = two$  in the program of Fig. 10.42. Then  $q$  can only be procedure one in line (4). Since  $change[one] = \{x\}$  initially, nothing is added to  $change[two]$  at line (5). At lines (6) and (7) we need consider only the second formal parameter of procedure one, since the first actual parameter is local to two. In the only call of one by two, the second actual parameter is  $y$ , and its corresponding formal,  $x$ , is changed, so we set  $change[two]$  to  $\{h, y\}$  at line (10).

We now consider  $p_i = one$ . At line (4),  $q$  can only be procedure two.





Fig. 10.44. Calling graph.

At line (5),  $h$  is a global in  $change[two]$ , so we set  $change[one] = \{h, x\}$ . At lines (6) and (7), only the first formal parameter of  $two$  is in  $change[two]$ , so we must add  $g$  and  $w$  to  $change[one]$  at line (10), these being the two first actual parameters in the calls of procedure  $two$ . Thus,  $change[one] = \{g, h, w, x\}$ .

Now consider  $main$ . Procedure  $one$  changes both its formals, so both  $h$  and  $i$  will be changed in the call of  $one$  by  $main$ . However,  $i$  is a local and need not be considered. Thus we set  $change[main] = \{g, h\}$ . Finally, we repeat the while-loop of line (2). On reconsidering  $two$ , we find that  $one$  modifies global  $g$ . Thus the call to  $one(k, y)$  causes  $g$  to be modified, so  $change[two] = \{g, h, y\}$ . No further changes occur in the iteration.  $\square$

### Use of Change Information

As an example of the way *change* can be used, consider the computation of global common subexpressions. Suppose we are computing available expressions for a procedure  $p$ , and we want to compute  $a\_kill[B]$  for a block  $B$ . A definition of variable  $a$  must be deemed to kill any expression involving  $a$  or involving some  $x$  that could be an alias of  $a$ . However, a call in  $B$  to procedure  $q$  cannot kill an expression involving  $a$  unless  $a$  is an alias (remember  $a$  is an alias of itself) of some variable in  $change[q]$ . Thus, the information computed by Algorithms 10.12 and 10.13 can be used to construct a safe approximation to the set of expressions killed.

To compute available expressions in programs with procedure calls, we must also have a conservative way of estimating the set of expressions generated by a procedure call. To be conservative, we may assume  $a+b$  is generated by a call to  $q$  if and only if, on every path from  $q$ 's entry to its return, we find  $a+b$  with no subsequent redefinition of  $a$  or  $b$ . When we look for occurrences of  $a+b$ , we must not accept  $x+y$  as such an occurrence unless we are sure that, in every call of  $q$ ,  $x$  is an alias of  $a$  and  $y$  an alias of  $b$ .

We make this requirement because it is conservative to err by assuming that an expression is not available when it is. By the same token, we must assume  $a+b$  is killed by a definition of any  $z$  that could possibly be an alias of  $a$  or  $b$ . Thus the simplest way to compute available expressions for all nodes of all

procedures is to assume that a call generates nothing, and that  $a\_kill[B]$  for all blocks  $B$  is as computed above. As one does not expect many expressions to be generated by the typical procedure, this approach is good enough for most purposes.

A more complicated, and more accurate, alternative approach to the computation of available expressions is to compute  $gen[p]$  for each procedure  $p$  iteratively. We may initialize  $gen[p]$  to be the set of expressions available at the end of  $p$ 's return node according to the method above. That is, no aliasing is permitted for generated expressions;  $a+b$  represents only itself, even if other variables could be aliases of  $a$  or  $b$ .

Now compute available expressions for all nodes of all procedures again. However, a call to  $q(a,b)$  generates those expressions in  $gen[q]$  with  $a$  and  $b$  substituted for the corresponding formal of  $q$ .  $a\_kill$  remains as before. A new value of  $gen[p]$ , for each procedure  $p$ , can be found by seeing what expressions are available at the end of  $p$ 's return. This iteration may be repeated until we get no more changes in available expressions at any node.

## 10.9 DATA-FLOW ANALYSIS OF STRUCTURED FLOW GRAPHS

Gotoless programs have reducible flow graphs; so do programs encouraged by several programming methodologies. Several studies of large classes of programs have revealed that virtually all programs written by people have flow graphs that are reducible.<sup>10</sup> This observation is relevant for optimization purposes because we can find optimization algorithms that run significantly faster on reducible flow graphs. In this section we discuss a variety of flow-graph concepts, such as "interval analysis," that are primarily relevant to structured flow graphs. In essence, we shall apply the syntax-directed techniques developed in Section 10.5 to the more general setting where the syntax doesn't necessarily provide the structure, but the flow graph does.

### Depth-First Search

There is a useful ordering of the nodes of a flow graph, known as *depth-first* ordering, which is a generalization of the depth-first traversal of a tree introduced in Section 2.3. A depth-first ordering can be used to detect loops in any flow graph; it also helps speed up iterative data-flow algorithms such as those discussed in Section 10.6. The depth-first ordering is created by starting at the initial node and searching the entire graph, trying to visit nodes as far away from the initial node as quickly as possible (*depth first*). The route of the search forms a tree. Before we give the algorithm, let us consider an example.

<sup>10</sup> "Written by people" is not redundant because we know of several programs that generate code with "rats' nests" of goto's. There is nothing wrong with this; the structure is in the input to these programs.

**Example 10.30.** One possible depth-first search of the flow graph in Fig. 10.45 is illustrated in Fig. 10.46. Solid edges form the tree; dashed edges are the other edges of the flow graph. The depth-first search of the flow graph corresponds to a preorder traversal of the tree,  $1 \rightarrow 3 \rightarrow 4 \rightarrow 6 \rightarrow 7 \rightarrow 8 \rightarrow 10$ , then back to 8, then to 9. We go back to 8 once more, retreating to 7, 6, and 4, and then forward to 5. We retreat from 5 back to 4, then back to 3 and 1. From 1 we go to 2, then retreat from 2, back to 1, and we have traversed the entire tree in preorder. Note that we have not yet explained how the tree is selected from the flow graph.  $\square$

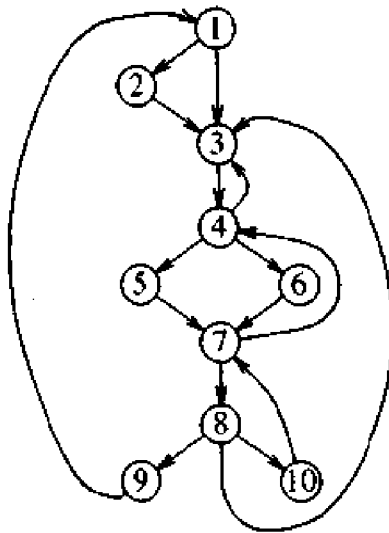


Fig. 10.45. Flow graph

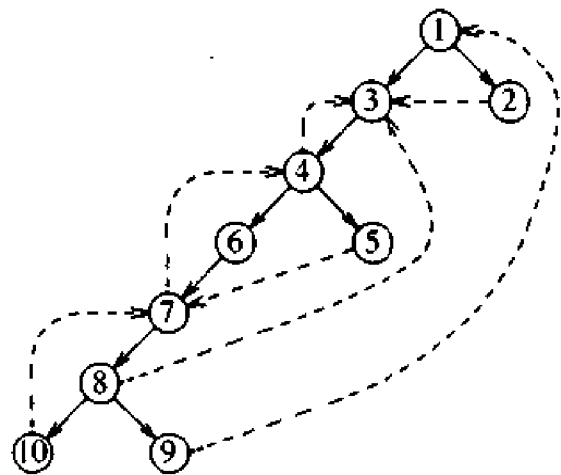


Fig. 10.46. Depth-first presentation

The *depth-first ordering* of the nodes is the reverse of the order in which we last visit the nodes in the preorder traversal.

**Example 10.31.** In Example 10.30, the complete sequence of nodes visited as we traverse the tree is

1, 3, 4, 6, 7, 8, 10, 8, 9, 8, 7, 6, 4, 5, 4, 3, 1, 2, 1.

In this list, mark the last occurrence of each number to get

1, 3, 4, 6, 7, 8, 10, 8, 9, 8, 7, 6, 4, 5, 4, 3, 1, 2, 1

The depth-first ordering is the sequence of marked numbers in reverse order. Here, this sequence happens to be 1, 2, ..., 10. That is, initially the nodes were numbered in depth-first order.  $\square$

We now give an algorithm that computes a depth-first ordering of a flow graph by constructing and traversing a tree rooted at the initial node, trying to make paths in the tree as long as possible. Such a tree is called a *depth-first spanning tree (dfst)*. It is this algorithm that was used to construct Fig. 10.46 from Fig. 10.45.

**Algorithm 10.14.** Depth-first spanning tree and depth-first ordering.

**Input.** A flow graph  $G$ .

**Output.** A *dfst*  $T$  of  $G$  and an ordering of the nodes of  $G$ .

**Method.** We use the recursive procedure  $search(n)$  of Fig. 10.47; the algorithm is to initialize all nodes of  $G$  to "unvisited," then call  $search(n_0)$ , where  $n_0$  is the initial node. When we call  $search(n)$ , we first mark  $n$  "visited," to avoid adding  $n$  to the tree twice. We use  $i$  to count from the number of nodes of  $G$  down to 1, assigning depth-first numbers  $dfn[n]$  to nodes  $n$  as we go. The set of edges  $T$  forms the depth-first spanning tree for  $G$ , and they are called *tree edges*.  $\square$

```

procedure  $search(n)$ ;
begin
(1)   mark  $n$  "visited";
(2)   for each successor  $s$  of  $n$  do
(3)       if  $s$  is "unvisited" then begin
(4)           add edge  $n \rightarrow s$  to  $T$ ;
(5)            $search(s)$ 
(6)       end;
(7)    $dfn[n] := i$ ;
(8)    $i := i - 1$ 
end;

/* main program follows */

(9)  $T := \text{empty}$ ; /* set of edges */
(10) for each node  $n$  of  $G$  do mark  $n$  "unvisited";
(11)  $i := \text{number of nodes of } G$ ;
(12)  $search(n_0)$ 

```

**Fig. 10.47.** Depth-first search algorithm.

**Example 10.32.** Consider Fig. 10.47. We set  $i$  to 10 and call  $search(1)$ . At line (2) of  $search$  we must consider each successor of node 1. Suppose we consider  $s = 3$  first. Then we add edge  $1 \rightarrow 3$  to the tree and call  $search(3)$ . In  $search(3)$  we add edge  $3 \rightarrow 4$  to  $T$  and call  $search(4)$ .

Suppose in  $search(4)$  we choose  $s = 6$  first. Then we add edge  $4 \rightarrow 6$  to  $T$  and call  $search(6)$ . This in turn causes us to add  $6 \rightarrow 7$  to  $T$  and call  $search(7)$ . Node 7 has two successors, 4 and 8. But 4 was already marked "visited" by  $search(4)$ , so we do nothing when  $s = 4$ . When  $s = 8$  we add edge  $7 \rightarrow 8$  to  $T$  and call  $search(8)$ . Suppose we then choose  $s = 10$ . We add edge  $8 \rightarrow 10$  and call  $search(10)$ .

Now 10 has a successor, 7, but 7 is already marked "visited," so in  $search(10)$ , we fall through to step (6) in Fig. 10.47, setting  $dfn[10] = 10$  and

$i = 9$ . This completes the call to *search*(10), so we return to *search*(8). We now set  $s = 9$  in *search*(8), add edge  $8 \rightarrow 9$  to  $T$  and call *search*(9). The only successor of 9, node 1, is already "visited," so we set  $dfn[9] = 9$  and  $i = 8$ . Then we return to *search*(8). The last successor of 8, node 3, is "visited," so we do nothing for  $s = 3$ . At this point, we have considered all successors of 8, so we set  $dfn[8] = 8$  and  $i = 7$ , returning to *search*(7).

All of 7's successors have been considered, so we set  $dfn[7] = 7$  and  $i = 6$ , returning to *search*(6). Similarly, 6's successors have been considered, so we set  $dfn[6] = 6$  and  $i = 5$ , and we return to *search*(4). Successor 3 of 4 has been "visited," but 5 has not, so we add  $4 \rightarrow 5$  to the tree and call *search*(5), which results in no further calls, as successor 7 of 5 has been "visited." Thus,  $dfn[5] = 5$ ,  $i$  is set to 4, and we return to *search*(4). We have completed consideration of the successors of 4, so we set  $dfn[4] = 4$  and  $i = 3$ , returning to *search*(3). Then we set  $dfn[3] = 3$  and  $i = 2$  and return to *search*(1).

The final steps are to call *search*(2) from *search*(1), set  $dfn[2] = 2$ ,  $i = 1$ , return to *search*(1), set  $dfn[1] = 1$  and  $i = 0$ . Note that we chose a numbering of the nodes such that  $dfn[i] = i$ , but that relation need not hold for an arbitrary graph, or even for another depth-first ordering of the graph of Fig. 10.45.  $\square$

### Edges in a Depth-First Presentation of a Flow Graph

When we construct a *dfst* for a flow graph, the edges of the flow graph fall into three categories.

1. There are edges that go from a node  $m$  to an ancestor of  $m$  in the tree (possibly to  $m$  itself). These edges we shall term *retreating* edges. For example,  $7 \rightarrow 4$  and  $9 \rightarrow 1$  are retreating edges in Fig. 10.46. It is an interesting and useful fact that if the flow graph is reducible, then the retreating edges are exactly the back edges of the flow graph,<sup>11</sup> independent of the order in which successors are visited in step (2) of Fig. 10.47. For any flow graph, every back edge is retreating, although if the graph is nonreducible there will be some retreating edges that are not back edges.
2. There are edges, called *advancing edges*, that go from a node  $m$  to a proper descendant of  $m$  in the tree. All edges in the *dfst* itself are advancing edges. There are no other advancing edges in Fig. 10.46, but, for example, if  $4 \rightarrow 8$  were an edge, it would be in this category.
3. There are edges  $m \rightarrow n$  such that neither  $m$  nor  $n$  is an ancestor of the other in the *dfst*. Edges  $2 \rightarrow 3$  and  $5 \rightarrow 7$  are the only such examples in Fig. 10.46. We call these edges *cross edges*. An important property of

<sup>11</sup> Recall, the back edges of a flow graph are those whose heads dominate their tails.

cross edges is that if we draw the *dfst* so children of a node are drawn from left to right in the order in which they were added to the tree, then all cross edges travel from right to left.

It should be noted that  $m \rightarrow n$  is a retreating edge if and only if  $dfn[m] \geq dfn[n]$ . To see why, note that if  $m$  is a descendant of  $n$  in the *dfst*, then  $search(m)$  terminates before  $search(n)$ , so  $dfn[m] \geq dfn[n]$ . Conversely, if  $dfn[m] \geq dfn[n]$ , then  $search(m)$  terminates before  $search(n)$ , or  $m = n$ . But  $search(n)$  must have begun before  $search(m)$  if there is an edge  $m \rightarrow n$ , or else the fact that  $n$  is a successor of  $m$  would have made  $n$  a descendant of  $m$  in the *dfst*. Thus the time  $search(m)$  is active is a subinterval of the time  $search(n)$  is active, from which it follows that  $n$  is an ancestor of  $m$  in the *dfst*.

### Depth of a Flow Graph

There is an important parameter of flow graphs called the *depth*. Given a depth-first spanning tree for the graph, the depth is the largest number of retreating edges on any cycle-free path.

**Example 10.33.** In Fig. 10.46, the depth is 3, since there is a path

$$10 \rightarrow 7 \rightarrow 4 \rightarrow 3$$

with three retreating edges, but no cycle-free path with four or more retreating edges. It is a coincidence that the “deepest” path here has only retreating edges; in general we may have a mixture of retreating, advancing, and cross edges in a “deepest” path.  $\square$

We can prove the depth is never greater than what one would intuitively call the depth of loop nesting in the flow graph. If a flow graph is reducible, we may replace “retreating” by “back” in the definition of “depth,” since the retreating edges in any *dfst* are exactly the back edges. The notion of depth then becomes independent of the *dfst* actually chosen.

### Intervals

The division of a flow graph into intervals serves to put a hierarchical structure on the flow graph. That structure in turn allows us to apply the rules for syntax-directed data-flow analysis whose development began in Section 10.5.

Intuitively, an “interval” in a flow graph is a natural loop plus an acyclic structure that dangles from the nodes of that loop. An important property of intervals is that they have *header* nodes that dominate all the nodes in the interval; that is, every interval is a region. Formally, given a flow graph  $G$  with initial node  $n_0$ , and a node  $n$  of  $G$ , the *interval with header  $n$* , denoted  $I(n)$ , is defined as follows.

1.  $n$  is in  $I(n)$ .
2. If all the predecessors of some node  $m \neq n_0$  are in  $I(n)$ , then  $m$  is in  $I(n)$ .
3. Nothing else is in  $I(n)$ .

We therefore may build  $I(n)$  by starting with  $n$ , and adding nodes  $m$  by rule (2). It does not matter in which order we add two candidates  $m$  because once a node's predecessors are all in  $I(n)$ , they remain in  $I(n)$ , and each candidate will eventually be added by rule (2). Eventually, no more nodes can be added to  $I(n)$ , and the resulting set of nodes is the interval with header  $n$ .

### Interval Partitions

Given any flow graph  $G$ , we can partition  $G$  into disjoint intervals as follows.

**Algorithm 10.15.** Interval analysis of a flow graph.

*Input.* A flow graph  $G$  with initial node  $n_0$ .

*Output.* A partition of  $G$  into a set of disjoint intervals.

*Method.* For any node  $n$ , we compute  $I(n)$  by the method sketched above:

```

 $I(n) := \{n\};$ 
while there exists a node  $m \neq n_0$ ,
    all of whose predecessors are in  $I(n)$  do
         $I(n) := I(n) \cup \{m\}$ 

```

The particular nodes that are headers of intervals in the partition are chosen as follows. Initially, no nodes are "selected."

```

construct  $I(n_0)$  and "select" all nodes in that interval;
while there is a node  $m$ , not yet "selected,"
    but with a selected predecessor do
        construct  $I(m)$  and "select" all nodes in that interval

```

□

Once a candidate  $m$  has a predecessor  $p$  selected,  $m$  can never be added to some interval not containing  $p$ . Thus, candidate  $m$ 's remain candidates until they are selected to head their own interval. Thus, the order in which interval headers  $m$  are picked in Algorithm 10.15 does not affect the final partition into intervals. Also, as long as all nodes are reachable from  $n_0$ , it can be shown by induction on the length of a path from  $n_0$  to  $n$  that node  $n$  will eventually either be put in some other node's interval, or will become a header of its own interval, but not both. Thus, the set of intervals constructed in Algorithm 10.15 truly partition  $G$ .

**Example 10.34.** Let us find the interval partition for Fig. 10.45. We start by constructing  $I(1)$ , because node 1 is the initial node. We can add 2 to  $I(1)$  because 2's only predecessor is 1. However, we cannot add 3 because it has predecessors, 4 and 8, that are not yet in  $I(1)$ , and similarly, every other node except 1 and 2 has predecessors not yet in  $I(1)$ . Thus,  $I(1) = \{1, 2\}$ .

We may now compute  $I(3)$  because 3 has some "selected" predecessors, 1 and 2, but 3 is not itself in an interval. However, no node can be added to  $I(3)$ , so  $I(3) = \{3\}$ . Now 4 is a header because it has a predecessor, 3, in an interval. We can add 5 and 6 to  $I(4)$  because these have only 4 as a

predecessor but no other nodes can be added; e.g., 7 has predecessor 10.

Next, 7 becomes a header, and we can add 8 to  $I(7)$ . Then, we can add 9 and 10, because these have only 8 as predecessor. Thus, the intervals in the partition of Fig. 10.45 are:

$$\begin{array}{ll} I(1) = \{1,2\} & I(4) = \{4,5,6\} \\ I(3) = \{3\} & I(7) = \{7,8,9,10\} \end{array} \quad \square$$

### Interval Graphs

From the intervals of one flow graph  $G$ , we can construct a new flow graph  $I(G)$  by the following rules.

1. The nodes of  $I(G)$  correspond to the intervals in the interval partition of  $G$ .
2. The initial node of  $I(G)$  is the interval of  $G$  that contains the initial node of  $G$ .
3. There is an edge from interval  $I$  to a different interval  $J$  if and only if in  $G$  there is an edge from some node in  $I$  to the header of  $J$ . Note that there could not be an edge entering some node  $n$  of  $J$  other than the header, from outside  $J$ , because then there would be no way  $n$  could have been added to  $J$  in Algorithm 10.15.

We may apply Algorithm 10.15 and the interval graph construction alternately, producing the sequence of graphs  $G, I(G), I(I(G)), \dots$ . Eventually, we shall come to a graph each of whose nodes is an interval all by itself. This graph is called the *limit flow graph* of  $G$ . It is an interesting fact that a flow graph is reducible if and only if its limit flow graph is a single node.<sup>12</sup>

**Example 10.35.** Fig. 10.48 shows the result of applying the interval construction repeatedly to Fig. 10.45. The intervals of that graph were given in Example 10.34, and the interval graph constructed from these is in Fig. 10.48(a). Note that the edge  $10 \rightarrow 7$  in Fig. 10.45 does not cause an edge from the node representing  $\{7,8,9,10\}$  to itself in Fig. 10.48(a), because the interval graph construction explicitly excluded such loops. Also note that the flow graph of Fig. 10.45 is reducible because its limit flow graph is a single node.  $\square$

### Node Splitting

If we reach a limit flow graph that is other than a single node, we can proceed further only if we split one or more nodes. If a node  $n$  has  $k$  predecessors, we may replace  $n$  by  $k$  nodes,  $n_1, n_2, \dots, n_k$ . The  $i$ th predecessor of  $n$  becomes the predecessor of  $n_i$  only, while all successors of  $n$  become

<sup>12</sup> In fact, this definition is historically the original definition.



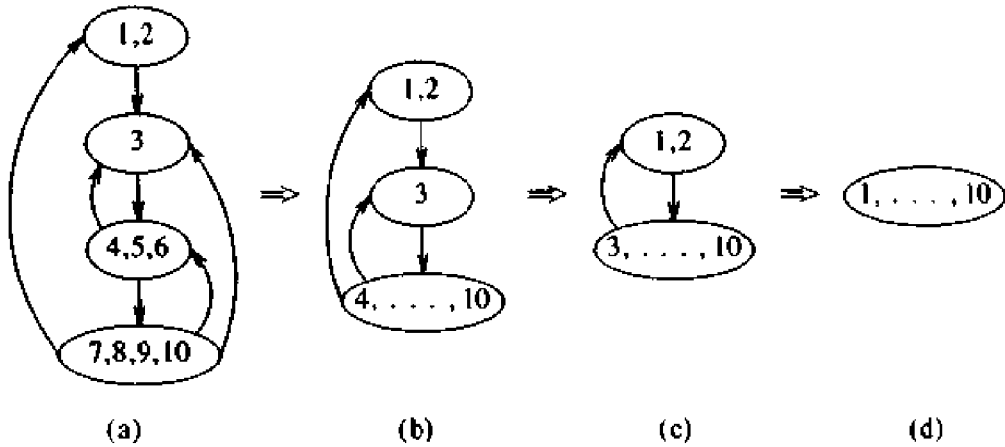


Fig. 10.48. Interval graph sequence.

successors of all of the  $n_i$ 's.

If we apply Algorithm 10.15 to the resulting graph, each  $n_i$  has a unique predecessor, and so it will surely become part of that predecessor's interval. Thus, one node splitting plus one round of interval partitioning results in a graph with fewer nodes. As a consequence, the construction of interval graphs, interspersed when necessary with node splitting, must eventually attain a graph of a single node. The significance of this observation will become clear in the next section, when we design data-flow analysis algorithms that are driven by these two operations on graphs.

**Example 10.36.** Consider the flow graph of Fig. 10.49(a), which is its own limit flow graph. We may split node 2 into  $2a$  and  $2b$ , with predecessors 1 and 3, respectively. This graph is shown in Fig. 10.49(b). If we apply interval partitioning twice, we get the sequence of graphs shown in Fig. 10.49(c) and (d), leading to a single node.  $\square$

### $T_1$ - $T_2$ Analysis

A convenient way to achieve the same effect as interval analysis is to apply two simple transformations to flow graphs.

$T_1$ : If  $n$  is a node with a loop, i.e., an edge  $n \rightarrow n$ , delete that edge.

$T_2$ : If there is a node  $n$ , not the initial node, that has a unique predecessor,  $m$ , then  $m$  may *consume*  $n$  by deleting  $n$  and making all successors of  $n$  (including  $m$ , possibly) be successors of  $m$ .

Some interesting facts about the  $T_1$  and  $T_2$  transformations are:

1. If we apply  $T_1$  and  $T_2$  to a flow graph  $G$  in any order, until a flow graph results for which no applications of  $T_1$  or  $T_2$  are possible, then a unique flow graph results. The reason is that a candidate for loop removal by  $T_1$

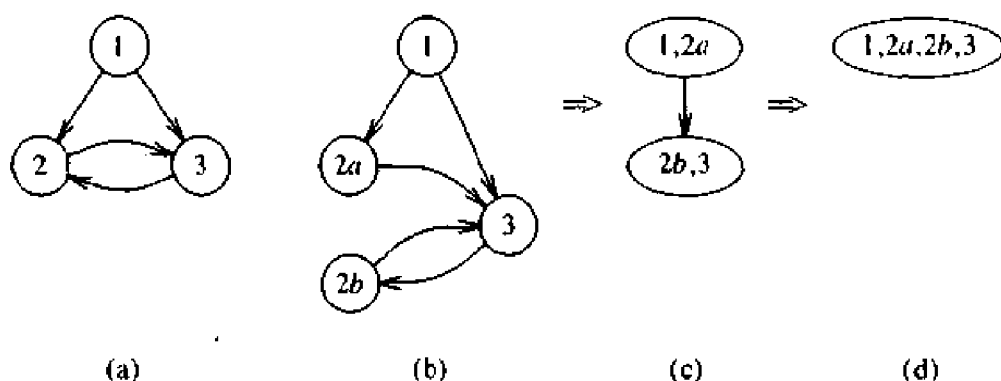


Fig. 10.49. Node splitting followed by interval partitioning.

or consumption by  $T_2$  remains a candidate, even if some other application of  $T_1$  or  $T_2$  is made first.

2. The flow graph resulting from exhaustive application of  $T_1$  and  $T_2$  to  $G$  is the limit flow graph of  $G$ . The proof is somewhat subtle and left as an exercise. As a consequence, another definition of "reducible flow graph" is one that can be converted to a single node by  $T_1$  and  $T_2$ .

**Example 10.37.** In Fig. 10.50 we see a sequence of  $T_1$  and  $T_2$  reductions starting from a flow graph that is a renaming of Fig. 10.49(b). In Fig. 10.50(b),  $c$  has consumed  $d$ . Note that the loop on  $cd$  in Fig. 10.50(b) results from the edge  $d \rightarrow c$  in Fig. 10.50(a). That loop is removed by  $T_1$  in Fig. 10.50(c). Also note that when  $a$  consumes  $b$  in Fig. 10.50(d), the edges from  $a$  and  $b$  to the node  $cd$  become a single edge.

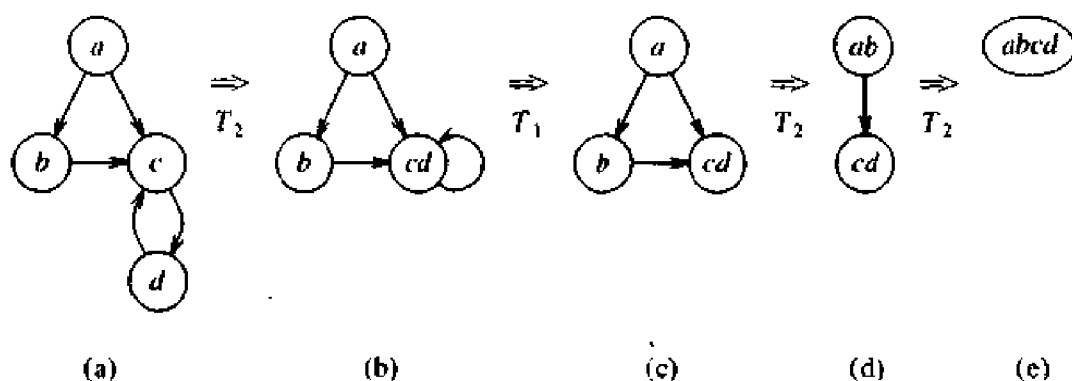


Fig. 10.50. Reduction by  $T_1$  and  $T_2$ .

## Regions

Recall from Section 10.5 that a region in a flow graph is a set of nodes  $N$  that includes a header, which dominates all the other nodes in a region. All edges between nodes in  $N$  are in the region, except (possibly) for some of those that enter the header. For example, every interval is a region, but there are regions that are not intervals because, for example, they may omit some nodes an interval would include, or they may omit some edges back to the header. There are also regions much larger than any interval, as we shall see.

As we reduce a flow graph  $G$  by  $T_1$  and  $T_2$ , at all times the following conditions are true:

1. A node represents a region of  $G$ .
2. An edge from  $a$  to  $b$  represents a set of edges. Each such edge is from some node in the region represented by  $a$  to the header of the region represented by  $b$ .
3. Each node and edge of  $G$  is represented by exactly one node or edge of the current graph.

To see why these observations hold, notice first that they hold trivially for  $G$  itself. Every node is a region by itself, and every edge represents only itself. Suppose we apply  $T_1$  to some node  $n$  representing a region  $R$ , while the loop  $n \rightarrow n$  represents some set of edges  $E$ , all of which must enter the header of  $R$ . If we add the edges  $E$  to region  $R$ , it is still a region, so after removing the edge  $n \rightarrow n$ , the node  $n$  represents  $R$  and the edges of  $E$ , which preserves conditions (1)–(3) above.

If we instead use  $T_2$  to consume node  $b$  by node  $a$ , let  $a$  and  $b$  represent regions  $R$  and  $S$  respectively. Also, let  $E$  be the set of edges represented by the edge  $a \rightarrow b$ . We claim  $R$ ,  $S$ , and  $E$  together form a region whose header is the header of  $R$ . To prove this, we must verify that the header of  $R$  dominates every node in  $S$ . If not, then there must be some path to the header of  $S$  that does not end with an edge of  $E$ . Then the last edge of this path would have to be represented in the current flow graph by some other edge entering  $b$ . But there can be no such edge, or  $T_2$  cannot be used to consume  $b$ .

**Example 10.38.** The node labeled  $cd$  in Fig. 10.50(b) represents the region shown in Fig. 10.51(a), which was formed by having  $c$  consume  $d$ . Note that the edge  $d \rightarrow c$  is not part of the region; in Fig. 10.50(b) that edge is represented by the loop on  $cd$ . However, in Fig. 10.50(c), the edge  $cd \rightarrow cd$  has been removed, and the node  $cd$  now represents the region shown in Fig. 10.51(b).

In Fig. 10.50(d), node  $cd$  still represents the region of Fig. 10.51(b), while node  $ab$  represents the region of Fig. 10.51(c). The edge  $ab \rightarrow cd$  in Fig. 10.50(d) represents the edges  $a \rightarrow c$  and  $b \rightarrow c$  of the original flow graph in Fig. 10.50(a). When we apply  $T_2$  to reach Fig. 10.50(e), the remaining node represents the entire flow graph, Fig. 10.50(a).  $\square$

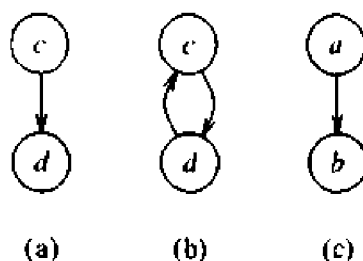


Fig. 10.51. Some regions.

We should observe that the property of  $T_1$  and  $T_2$  reduction mentioned above holds also for interval analysis. We leave as an exercise the fact that as we construct  $I(G)$ ,  $I(I(G))$ , and so on, each node in each of these graphs represents a region, and each edge a set of edges satisfying property (2) above.

### Finding Dominators

We close this section with an efficient algorithm for a concept that we have used frequently, and will continue to use in developing the theory of flow graphs and data-flow analysis. We shall give a simple algorithm for computing the dominators of every node  $n$  in a flow graph, based on the principle that if  $p_1, p_2, \dots, p_k$  are all the predecessors of  $n$ , and  $d \neq n$ , then  $d \text{ dom } n$  if and only if  $d \text{ dom } p_i$  for each  $i$ . The method is akin to forward data-flow analysis with intersection as the confluence operator (e.g., available expressions), in that we take an approximation to the set of dominators of  $n$  and refine it by repeatedly visiting all the nodes in turn.

In this case, the initial approximation we choose has the initial node dominated only by the initial node, and everything dominating everything besides the initial node. Intuitively, the reason this approach works is that dominator candidates are ruled out only when we find a path that proves, say,  $m \text{ dom } n$  is false. If we cannot find such a path, from the initial node to  $n$  avoiding  $m$ , then  $m$  really is a dominator of  $n$ .

#### Algorithm 10.16. Finding dominators.

*Input.* A flow graph  $G$  with set of nodes  $N$ , set of edges  $E$  and initial node  $n_0$ .

*Output.* The relation  $\text{dom}$ .

*Method.* We compute  $D(n)$ , the set of dominators of  $n$ , iteratively by the procedure in Fig. 10.52. At the end,  $d$  is in  $D(n)$  if and only if  $d \text{ dom } n$ . The reader may supply the details regarding how changes to  $D(n)$  are detected; Algorithm 10.2 will serve as a model.

One can show that  $D(n)$  computed at line (5) of Fig. 10.52 is always a subset of the current  $D(n)$ . Since  $D(n)$  cannot get smaller indefinitely, we must eventually terminate the while-loop. A proof that, after convergence,  $D(n)$  is

```

(1)  $D(n_0) := \{n_0\};$ 
(2) for  $n$  in  $N - \{n_0\}$  do  $D(n) := N;$ 
    /* end initialization */
(3) while changes to any  $D(n)$  occur do
(4)     for  $n$  in  $N - \{n_0\}$  do
(5)          $D(n) := \{n\} \cup \bigcap_{\substack{p \text{ a pred-} \\ \text{cessor of } n}} D(p);$ 

```

Fig. 10.52. Dominator computing algorithm.

the set of dominators of  $n$  is left for the interested reader. The algorithm of Fig. 10.52 is quite efficient, as  $D(n)$  can be represented by a bit vector and the set operations of line (5) can be done with logical **and** and **or**.  $\square$

**Example 10.39.** Let us return to the flow graph of Fig. 10.45, and suppose in the for-loop of line (4) nodes are visited in numerical order. Node 2 has only 1 for a predecessor, so  $D(2) := \{2\} \cup D(1)$ . Since 1 is the initial node,  $D(1)$  was assigned  $\{1\}$  at line (1). Thus,  $D(2)$  is set to  $\{1, 2\}$  at line (5).

Then node 3, with predecessors 1, 2, 4, and 8, is considered. Line (5) gives us  $D(3) = \{3\} \cup (\{1\} \cap \{1, 2\} \cap \{1, 2, \dots, 10\}) = \{1, 3\}$ . The remaining calculations are:

$$D(4) = \{4\} \cup (D(3) \cap D(7)) = \{4\} \cup (\{1, 3\} \cap \{1, 2, \dots, 10\}) = \{1, 3, 4\}$$

$$D(5) = \{5\} \cup D(4) = \{5\} \cup \{1, 3, 4\} = \{1, 3, 4, 5\}$$

$$D(6) = \{6\} \cup D(4) = \{6\} \cup \{1, 3, 4\} = \{1, 3, 4, 6\}$$

$$\begin{aligned} D(7) &= \{7\} \cup (D(5) \cap D(6) \cap D(10)) \\ &= \{7\} \cup (\{1, 3, 4, 5\} \cap \{1, 3, 4, 6\} \cap \{1, 2, \dots, 10\}) = \{1, 3, 4, 7\} \end{aligned}$$

$$D(8) = \{8\} \cup D(7) = \{8\} \cup \{1, 3, 4, 7\} = \{1, 3, 4, 7, 8\}$$

$$D(9) = \{9\} \cup D(8) = \{9\} \cup \{1, 3, 4, 7, 8\} = \{1, 3, 4, 7, 8, 9\}$$

$$D(10) = \{10\} \cup D(8) = \{10\} \cup \{1, 3, 4, 7, 8\} = \{1, 3, 4, 7, 8, 10\}$$

The second pass through the while-loop is seen to produce no changes, so the above values yield the relation *dom*.  $\square$

## 10.10 EFFICIENT DATA-FLOW ALGORITHMS

In this section we shall consider two ways to use flow-graph theory to speed data-flow analysis. The first is an application of depth-first ordering to reduce the number of passes that the iterative algorithms of Sections 10.6 take, and the second uses intervals or the  $T_1$  and  $T_2$  transformations to generalize the syntax-directed approach of Section 10.5.

### Depth-First Ordering in Iterative Algorithms

In all the problems studied so far, such as reaching definitions, available expressions, or live variables, any event of significance at a node will be propagated to that node along an acyclic path. For example, if a definition  $d$  is in  $in[B]$ , then there is some acyclic path from the block containing  $d$  to  $B$  such that  $d$  is in the  $in$ 's and  $out$ 's all along that path. Similarly, if an expression  $x+y$  is not available at the entrance to block  $B$ , then there is some acyclic path that demonstrates that fact; either the path is from the initial node and includes no statement that kills or generates  $x+y$ , or the path is from a block that kills  $x+y$  and along the path there is no subsequent generation of  $x+y$ . Finally, for live variables, if  $x$  is live on exit from block  $B$ , then there is an acyclic path from  $B$  to a use of  $x$ , along which there are no definitions of  $x$ .

The reader should check that in each of these cases, paths with cycles add nothing. For example, if a use of  $x$  is reached from the end of block  $B$  along a path with a cycle, we can eliminate that cycle to find a shorter path along which the use of  $x$  is still reached from  $B$ .

If all useful information propagates along acyclic paths, we have an opportunity to tailor the order in which we visit nodes in iterative data-flow algorithms so that after relatively few passes through the nodes, we can be sure information has passed along all the acyclic paths. In particular, statistics gathered in Knuth [1971b] show that typical flow graphs have a very low *interval depth*, which is the number of times one must apply the interval partition to reach the limit flow graph; an average of 2.75 was found. Furthermore, it can be shown that the interval depth of a flow graph is never less than what we have called the "depth," the maximum number of retreating edges on any acyclic path. (If the flow graph is not reducible, the depth may depend on the depth-first spanning tree chosen.)

Recalling our discussion of the depth-first spanning tree in the previous section, we note that if  $a \rightarrow b$  is an edge, then the depth-first number of  $b$  is less than that of  $a$  only when the edge is a retreating edge. Thus, replace line (5) of Fig. 10.26, which tells us to visit each block  $B$  of the flow graph for which we are computing reaching definitions, by:

**for each block  $B$  in depth-first order do**

Suppose we have a path along which a definition  $d$  propagates, such as

$3 \rightarrow 5 \rightarrow 19 \rightarrow 35 \rightarrow 16 \rightarrow 23 \rightarrow 45 \rightarrow 4 \rightarrow 10 \rightarrow 17$

where integers represent the depth-first numbers of the blocks along the path. Then the first time through the loop of lines (5)–(9) in Fig. 10.26,  $d$  will propagate from  $out[3]$  to  $in[5]$  to  $out[5]$ , and so on, up to  $out[35]$ . It will not reach  $in[16]$  on that round, because as 16 precedes 35, we had already computed  $in[16]$  by the time  $d$  was put in  $out[35]$ . However, the next time we run through the loop of lines (5)–(9), when we compute  $in[16]$ ,  $d$  will be included because it is in  $out[35]$ . Definition  $d$  will also propagate to  $out[16]$ ,  $in[23]$ , and so on, up to  $out[45]$ , where it must wait because  $in[4]$  was already computed.

On the third pass,  $d$  travels to  $in[4]$ ,  $out[4]$ ,  $in[10]$ ,  $out[10]$ , and  $in[17]$ , so after three passes we establish that  $d$  reaches block 17.<sup>13</sup>

It should not be hard to extract the general principle from this example. If we use depth-first order in Fig. 10.26, then the number of passes needed to propagate any reaching definition along any acyclic path is no more than one greater than the number of edges along that path that go from a higher numbered block to a lower numbered block. Those edges are exactly the retreating edges, so the number of passes needed is one plus the depth. Of course Algorithm 10.2 does not detect the fact that all definitions have reached wherever they can reach for one more pass, so the upper bound on the number of passes taken by that algorithm with depth-first block ordering is actually two plus the depth, or 5 if we believe the results of Knuth [1971b] to be typical.

The depth-first order is also advantageous for available expressions (Algorithm 10.3), or any data flow problem that we solved by propagation in the forward direction. For problems like live variables, where we propagate backwards, the same average of five passes can be achieved if we chose the reverse of the depth-first order. Thus, we may propagate a use of a variable in block 17 backwards along the path

$$3 \rightarrow 5 \rightarrow 19 \rightarrow 35 \rightarrow 16 \rightarrow 23 \rightarrow 45 \rightarrow 4 \rightarrow 10 \rightarrow 17$$

in one pass to  $in[4]$ , where we must wait for the next pass to in order reach  $out[45]$ . On the second pass it reaches  $in[16]$ , and on the third pass it goes from  $out[35]$  to  $out[3]$ . In general, one plus the depth suffices to carry the use of a variable backward, along any acyclic path, if we choose the reverse of depth-first order to visit the nodes in a pass, because then, uses propagate along any decreasing sequence in a single pass.

### Structure-Based Data-Flow Analysis

With a bit more effort, we can implement data-flow algorithms that visit nodes (and apply data-flow equations) no more times than the interval depth of the flow graph, and frequently the average node will be visited even fewer times than that. Whether the extra effort results in a true time savings has not been firmly established, but a technique like this one, based on interval analysis, has been used in several compilers. Further, the ideas exposed here apply to syntax-directed data-flow algorithms for all sorts of structured control statements, not just the **if**  $\cdots$  **then** and **do**  $\cdots$  **while** discussed in Section 10.5, and these have also appeared in several compilers.

We shall base our algorithm on the structure induced on flow graphs by the  $T_1$  and  $T_2$  transformations. As in Section 10.5, we are concerned with the definitions that are generated and killed as control flows through a region. Unlike the regions defined by **if** or **while** statements, a general region can have multiple exits, so for each block  $B$  in region  $R$  we shall compute sets

<sup>13</sup> Definition  $d$  also reaches  $out[17]$ , but that is irrelevant to the path in question.

$gen_{R,B}$  and  $kill_{R,B}$  of definitions generated and killed, respectively, along paths within the region from the header to the end of block  $B$ . These sets will be used to define a *transfer function*  $trans_{R,B}(S)$  that tells for any set  $S$  of definitions, what set of definitions reach the end of block  $B$  by traveling along paths wholly within  $R$ , given that all and only the definitions in  $S$  reach the header of  $R$ .

As we have seen in Sections 10.5 and 10.6, the definitions reaching the end of block  $B$  fall into two classes.

1. Those that are generated within  $R$  and propagate to the end of  $B$  independent of  $S$ .
2. Those that are not generated in  $R$ , but that also are not killed along some path from the header of  $R$  to the end of  $B$ , and therefore are in  $trans_{R,B}(S)$  if and only if they are in  $S$ .

Thus, we may write  $trans$  in the form:

$$trans_{R,B}(S) = gen_{R,B} \cup (S - kill_{R,B})$$

The heart of the algorithm is a way to compute  $trans_{R,B}$  for progressively larger regions defined by some  $(T_1, T_2)$ -decomposition of a flow graph. For the moment, we assume that the flow graph is reducible, although a simple modification allows the algorithm to work for nonreducible graphs as well.

The basis is a region consisting of a single block,  $B$ . Here the transfer function of the region is the transfer function of the block itself, since a definition reaches the end of the block if and only if it is either generated by the block or is in the set  $S$  and not killed. That is,

$$gen_{B,B} = gen[B]$$

$$kill_{B,B} = kill[B]$$

Now, let us consider the construction of a region  $R$  by  $T_2$ ; that is,  $R$  is formed when  $R_1$  consumes  $R_2$ , as suggested in Fig. 10.53. First, note that within region  $R$  there are no edges from  $R_2$  back to  $R_1$  since any edge from  $R_2$  to the header of  $R_1$  is not a part of  $R$ . Thus any path totally within  $R$  goes (optionally) through  $R_1$  first, then (optionally) through  $R_2$ , but cannot then return to  $R_1$ . Also note that the header of  $R$  is the header of  $R_1$ . We may conclude that within  $R$ ,  $R_2$  does not affect the transfer function of nodes in  $R_1$ ; that is,

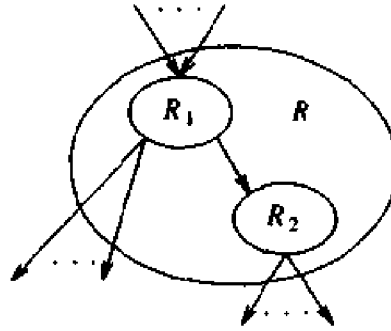
$$gen_{R,B} = gen_{R_1,B}$$

$$kill_{R,B} = kill_{R_1,B}$$

for all  $B$  in  $R_1$ .

For  $B$  in  $R_2$ , a definition can reach the end of  $B$  if any of the following conditions hold.



Fig. 10.53. Region building by  $T_2$ .

1. The definition is generated within  $R_2$ .
2. The definition is generated within  $R_1$ , reaches the end of some predecessor of the header of  $R_2$ , and is not killed going from the header of  $R_2$  to  $B$ .
3. The definition is in the set  $S$  available at the header of  $R_1$ , not killed going to some predecessor of the header of  $R_2$ , and not killed going from the header of  $R_2$  to  $B$ .

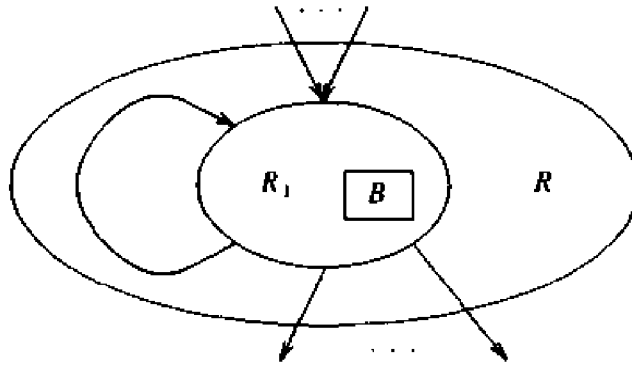
Hence, the definitions reaching the ends of those blocks in  $R_1$  that are predecessors of the header of  $R_2$  play a special role. In essence, we see what happens to a set  $S$  entering the header of  $R_1$  as its definitions try to reach the header of  $R_2$ , via one of its predecessors. The set of definitions that reach one of the predecessors of the header of  $R_2$  becomes the input set for  $R_2$ , and we apply the transfer functions for  $R_2$  to that set.

Thus, let  $G$  be the union of  $gen_{R_1,P}$  for all predecessors  $P$  of the header of  $R_2$ , and let  $K$  be the intersection of  $kill_{R_1,P}$  for all those predecessors  $P$ . Then if  $S$  is the set of definitions that reach the header of  $R_1$ , the set of definitions that reach the header of  $R_2$  along paths staying wholly within  $R$  is  $G \cup (S - K)$ . Therefore, the transfer function in  $R$  for those blocks  $B$  in  $R_2$  may be computed by

$$gen_{R,B} = gen_{R_2,B} \cup (G - kill_{R_2,B})$$

$$kill_{R,B} = kill_{R_2,B} \cup (K - gen_{R_2,B})$$

Next, consider what happens when a region  $R$  is built from a region  $R_1$  using transformation  $T_1$ . The general situation is shown in Fig. 10.54; note that  $R$  consists of  $R_1$  plus some back edges to the header of  $R_1$  (which is also the header of  $R$ , of course). A path going through the header twice would be cyclic and, as we argued earlier in the section, need not be considered. Thus, all definitions generated at the end of block  $B$  are generated in one of two ways.

Fig. 10.54. Region building by  $T_1$ .

1. The definition is generated within  $R_1$  and does not need the back edges incorporated into  $R$  in order to reach the end of  $B$ .
2. The definition is generated somewhere within  $R_1$ , reaches a predecessor of the header, follows one back edge, and is not killed going from the header to  $B$ .

If we let  $G$  be the union of  $gen_{R_1,P}$  for all predecessors of the header in  $R$ , then

$$gen_{R,B} = gen_{R_1,B} \cup (G - kill_{R_1,B})$$

A definition is killed going from the header to  $B$  if and only if it is killed along all acyclic paths, so the back edges incorporated into  $R$  do not cause more definitions to be killed. That is,

$$kill_{R,B} = kill_{R_1,B}$$

**Example 10.40.** Let us reconsider the flow graph of Fig. 10.50, whose  $(T_1, T_2)$ -decomposition is shown in Fig. 10.55, with the regions of the decomposition named. We also show in Fig. 10.56 some hypothetical bit vectors representing three definitions and whether they are generated or killed by each of the blocks in Fig. 10.55.

Starting from the inside out, we note that for single-node regions, which we call  $A$ ,  $B$ ,  $C$ , and  $D$ ,  $gen$  and  $kill$  are given by the table in Fig. 10.56. We may then proceed to region  $R$ , which is formed when  $C$  consumes  $D$  by  $T_2$ . Following the rules for  $T_2$  above, we note that  $gen$  and  $kill$  do not change for  $C$ , that is,

$$\begin{aligned} gen_{R,C} &= gen_{C,C} = 000 \\ kill_{R,C} &= kill_{C,C} = 010 \end{aligned}$$

For node  $D$ , we have to find in region  $C$  the union of  $gen$  for all the predecessors of the header of the region  $D$ . Of course the header of region  $D$  is node

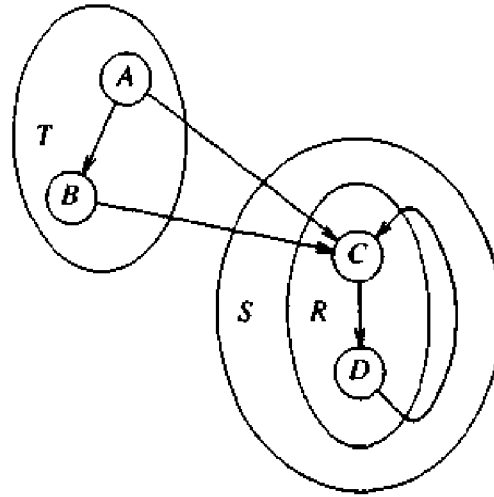


Fig. 10.55. Decomposition of a flow graph.

| BLOCK | <i>gen</i> | <i>kill</i> |
|-------|------------|-------------|
| A     | 100        | 010         |
| B     | 010        | 101         |
| C     | 000        | 010         |
| D     | 001        | 000         |

Fig. 10.56. *gen* and *kill* information for blocks in Fig. 10.55.

*D*, and there is only one predecessor of that node in region *C*, namely, the node *C*. Thus,

$$\begin{aligned} gen_{R,D} &= gen_{D,D} \cup (gen_{C,C} - kill_{D,D}) = 001 + (000 - 000) = 001 \\ kill_{R,D} &= kill_{D,D} \cup (kill_{C,C} - gen_{D,D}) = 000 + (010 - 001) = 010 \end{aligned}$$

Now, we build region *S* from region *R* by  $T_1$ . The *kill*'s don't change, so we have

$$\begin{aligned} kill_{S,C} &= kill_{R,C} = 010 \\ kill_{S,D} &= kill_{R,D} = 010 \end{aligned}$$

To compute the *gen*'s for *S* we note that the only back edge to the header of *S* that is incorporated going from *R* to *S* is the edge  $D \rightarrow C$ . Thus,

$$\begin{aligned} gen_{S,C} &= gen_{R,C} \cup (gen_{R,D} - kill_{R,C}) = 000 + (001 - 010) = 001 \\ gen_{S,D} &= gen_{R,D} \cup (gen_{R,D} - kill_{R,D}) = 001 + (001 - 010) = 001 \end{aligned}$$

The computation for region *T* is analogous to that for region *R* and we obtain

$$\begin{aligned}
gen_{T,A} &= 100 \\
kill_{T,A} &= 010 \\
gen_{T,B} &= 010 \\
kill_{T,B} &= 101
\end{aligned}$$

Finally, we compute *gen* and *kill* for region *U*, the entire flow graph. Since *U* is constructed when *T* consumes *S* by transformation  $T_2$ , the values of *gen* and *kill* for nodes *A* and *B* do not change from what was just given above. For *C* and *D*, we note that the header of *S*, node *C*, has two predecessors in region *T*, namely *A* and *B*. Therefore, we compute

$$\begin{aligned}
G &= gen_{T,A} \cup gen_{T,B} = 110 \\
K &= kill_{T,A} \cap kill_{T,B} = 000
\end{aligned}$$

Then we may compute

$$\begin{aligned}
gen_{U,C} &= gen_{S,C} \cup (G - kill_{S,C}) = 101 \\
kill_{U,C} &= kill_{S,C} \cup (K - gen_{S,C}) = 010 \\
gen_{U,D} &= gen_{S,D} \cup (G - kill_{S,D}) = 101 \\
kill_{U,D} &= kill_{S,D} \cup (K - gen_{S,D}) = 010
\end{aligned}$$

□

Having computed  $gen_{U,B}$  and  $kill_{U,B}$  for each block *B*, where *U* is the region consisting of the entire flow graph, we essentially have computed *out*[*B*] for each block *B*. That is, if we look at the definition of  $trans_{U,B}(S) = gen_{U,B} \cup (S - kill_{U,B})$ , we note that  $trans_{U,B}(\emptyset)$  is exactly *out*[*B*]. But  $trans_{U,B}(\emptyset) = gen_{U,B}$ . Thus, the completion of the structure-based reaching definition algorithm is to use the *gen*'s as the *out*'s, and compute the *in*'s by taking the union of the *out*'s of the predecessors. These steps are summarized in the following algorithm.

**Algorithm 10.17.** Structure-based reaching definitions.

*Input.* A reducible flow graph *G* and sets of definitions *gen*[*B*] and *kill*[*B*] for each block *B* of *G*.

*Output.* *in*[*B*] for each block *B*.

*Method.*

1. Find the  $(T_1, T_2)$ -decomposition for *G*.
2. For each region *R* in the decomposition, from the inside out, compute  $gen_{R,B}$  and  $kill_{R,B}$  for each block *B* in *R*.
3. If *U* is the name of the region consisting of the entire graph, then for each block *B*, set *in*[*B*] to the union, over all predecessors *P* of block *B*, of  $gen_{U,P}$ . □

### Some Speedups to the Structure-Based Algorithm

First, notice that if we have a transfer function  $G \cup (S - K)$ , the function is not changed if we delete from  $K$  some members of  $G$ . Thus, when we apply  $T_2$ , instead of using the formulas

$$gen_{R,B} = gen_{R_2,B} \cup (G - kill_{R_2,B})$$

$$kill_{R,B} = kill_{R_2,B} \cup (K - gen_{R_2,B})$$

we can replace the second by

$$kill_{R,B} = kill_{R_2,B} \cup K$$

thus saving an operation for each block in region  $R_2$ .

Another useful idea is to notice that the only time we apply  $T_1$  is after we have first consumed some region  $R_2$  by  $R_1$ , and there are some back edges from  $R_2$  to the header of  $R_1$ . Instead of first making changes in  $R_2$  because of the  $T_2$  operation, and then making changes in  $R_1$  and  $R_2$  due to the  $T_1$  operation, we can combine the two sets of changes if we do the following.

1. Using the  $T_2$  rule, compute the new transfer function for those nodes in  $R_2$  that are predecessors of the header of  $R_1$ .
2. Using the  $T_1$  rule, compute the new transfer function for all the nodes of  $R_1$ .
3. Using the  $T_2$  rule, compute the new transfer function for all the nodes of  $R_2$ . Note that feedback due to the application of  $T_1$  has reached the predecessors of  $R_2$  and is passed to all of  $R_2$  by the  $T_2$  rule; there is no need to apply the  $T_1$  rule for  $R_2$ .

### Handling Nonreducible Flow Graphs

If the  $(T_1, T_2)$ -reduction of a flow graph stops at a limit flow graph that is not a single node, then we must perform a node splitting. Splitting a node of the limit flow graph corresponds to duplicating the entire region represented by that node. For example, in Fig. 10.57 we suggest the effect that node splitting might have on an original nine-node flow graph that was partitioned by  $T_1$  and  $T_2$  into three regions connected by some edges.

As mentioned in the previous section, by alternating splits with sequences of reductions, we are guaranteed to reduce the flow graph to a single node. The result of the splits is that some of the nodes of the original graph will have more than one copy in the region represented by the one-node graph. We may apply Algorithm 10.17 to this region with little change. The only difference is that when we split a node, the *gen*'s and *kill*'s for the nodes of the original graph in the region represented by the split node must be duplicated. For example, whatever the values of *gen* and *kill* are for the nodes in the two-node region of Fig. 10.57 on the left become *gen* and *kill* for each of the corresponding nodes in both two-node regions on the right. At the final step,

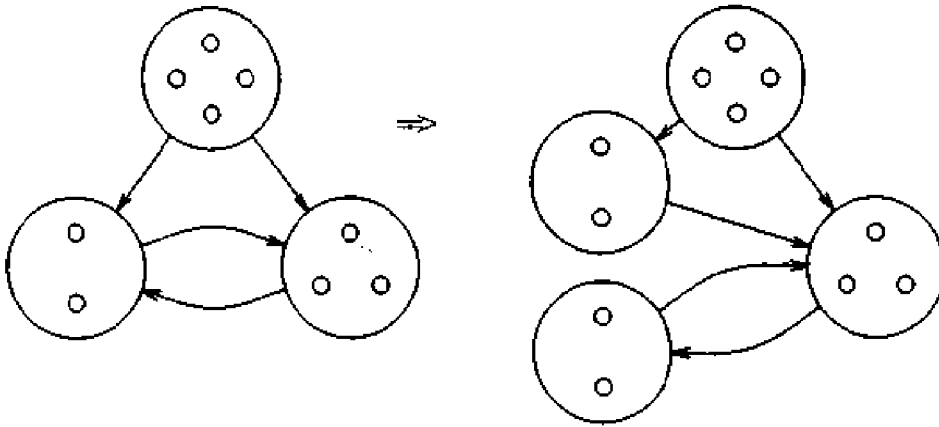


Fig. 10.57. Splitting a nonreducible flow graph.

when we compute the *in*'s for all the nodes, those nodes of the original graph that have several representatives in the final region have their *in*'s computed by taking the union of the *in*'s of all their representatives.

In the worst case, splitting of nodes could exponentiate the total number of nodes represented by all the regions. Thus, if we expect many flow graphs to be nonreducible, we probably should not use structure-based methods. Fortunately, nonreducible flow graphs are sufficiently rare that we can generally ignore the cost of node splitting.

## 10.11 A TOOL FOR DATA-FLOW ANALYSIS

As we have pointed out before, there are strong similarities among the various data-flow problems studied. The data-flow equations of Section 10.6 were seen to be distinguished by:

1. The transfer function used, which in each case studied was of the form  $f(X) = A \cup (X - B)$ . For example,  $A = \text{kill}$  and  $B = \text{gen}$  for reaching definitions.
2. The confluence operator, which in all cases so far has been either union or intersection.
3. The direction of propagation of information: forward or backward.

Since these distinctions are not great, it should not be surprising that all these problems can be treated in a unified way. Such an approach was described in Kildall [1973], and a tool to simplify the implementation of data-flow problems was implemented by Kildall and used by him in several compiler projects. It has not seen widespread use, probably because the amount of labor saved by the system is not as great as that saved by tools like parser generators. However, we should be aware of what can be done not only because it does suggest a simplification for implementers of optimizing

compilers, but also because it helps to unify the various ideas we have seen so far in this chapter. Further, this section suggests how more powerful data-flow analysis strategies, ones that provide more precise information than the algorithms mentioned so far, can be developed.

### Data-Flow Analysis Frameworks

We shall describe frameworks that model forward propagation problems. If we consider only the iterative type of solution to data-flow problems, then the direction of flow makes no difference; we can reverse the direction of edges and make some minor adjustments to account for the initial node, and then treat a backward problem as if it were forward. Structure-based algorithms are somewhat different; the forward and backward problems are not solved in quite the same way because the reverse of a reducible flow graph need not be reducible. However, the treatment of backward problems will be left as an exercise, and we shall restrict ourselves to forward problems only.

A *data-flow analysis framework* consists of:

1. A set  $V$  of *values* to be propagated. The values of *in* and *out* are members of  $V$ .
2. A set  $F$  of *transfer functions* from  $V$  to  $V$ .
3. A binary *meet operation*  $\wedge$ , on  $V$ , to represent the confluence operator.

**Example 10.41.** For reaching definitions,  $V$  consists of all subsets of the set of definitions in the program. The set  $F$  is the set of all functions of the form  $f(X) = A \cup (X - B)$ , where  $A$  and  $B$  are sets of definitions, i.e., members of  $V$ ;  $A$  and  $B$  are what we called *gen* and *kill*, respectively. Finally, the operation  $\wedge$  is union.

For available expressions,  $V$  consists of all subsets of the set of expressions computed by the program, and  $F$  is the set of expressions of the same form as above, but where  $A$  and  $B$  are now sets of expressions. The meet operation is intersection, of course.  $\square$

**Example 10.42.** Kildall's approach is not limited to the simple examples with which we have been dealing, although the complexity, both in terms of computation time and intellectual difficulty, rises. The exercises suggest a very powerful example, where the data-flow information computed tells us in essence, all pairs of expressions that have the same value at a point. However, we shall get some of the flavor of this example by giving a method for telling which variables have constant values in a way that captures more information than reaching definitions. Our new framework understands, for example, that when  $x$  is defined by  $d: x := x + 1$ , and  $x$  had a constant value before assignment, it does so afterward.

In contrast, using reaching definitions for constant propagation we would see that statement  $d$  was a possible definition of  $x$ , and assume therefore that  $x$  did not have a constant value. Of course, in one pass the right side of

$d: x := x + 1$  might be replaced by a constant, and then another round of constant propagation could detect that uses of the  $x$  defined at  $d$  were actually uses of a constant.

In the new framework, the set  $V$  is the set of all mappings from the variables of the program to a particular set of values. That set of values consists of

1. All constants.
2. The value *nonconst*, which means that the variable in question has been determined not to have a constant value. The value *nonconst* would be assigned to variable  $x$  if, say, during data-flow analysis we discovered two paths along which the values 2 and 3, respectively, were assigned to  $x$ , or a path along which the previous definition of  $x$  was a read statement.
3. The value *undef*, which means that nothing may yet be asserted about the variable in question, presumably because early in the execution of the data-flow analysis, no definition of the variable has been discovered to reach the point in question.

Note that *nonconst* and *undef* are not the same; they are essentially opposites. The first says we have seen so many ways a variable could be defined that we know it is not constant. The second says we have seen so little about the variable, that we cannot say anything at all.

The meet operation is defined by the following table. We let  $\mu$  and  $\nu$  be two members of  $V$ ; that is,  $\mu$  and  $\nu$  both map each variable to a constant, to *undef*, or to *nonconst*. Then the function  $\rho = \mu \wedge \nu$  is defined in Fig. 10.58, where we give the value of  $\rho(x)$  in terms of the values of  $\mu(x)$  and  $\nu(x)$  for each variable  $x$ . In that table,  $c$  is an arbitrary constant, and  $d$  is another constant definitely not equal to  $c$ . For example, if  $\mu(x) = c$  and  $\nu(x) = d$ , a different constant, then apparently  $x$  takes the values  $c$  and  $d$  along two different paths, and at the confluence of those paths,  $x$  has no constant value; hence the choice  $\rho(x) = \text{nonconst}$ . As another example, if along one path, nothing is known about  $x$ , reflected by  $\mu(x) = \text{undef}$ , and along another path,  $x$  is believed to have the value  $c$ , then after the confluence of these paths, we can only assert that  $x$  has the value  $c$ . Of course, later discovery of another path to the point of confluence, along which  $x$  has a value besides  $c$ , will change the assigned value for  $x$  after the confluence to *nonconst*.

Finally, we must design the set of functions  $F$  that reflect the transfer of information from the beginning to the end of any block. The description of this set of functions is complicated, although the ideas are straightforward. We shall therefore give a "basis" for the set of functions by describing the functions that represent single definition statements, and the entire set of functions can then be constructed by composing functions from this basis set to reflect blocks with more than one definition statement.



| $v(x)$          | $\mu(x)$        |                 |                       |                 |
|-----------------|-----------------|-----------------|-----------------------|-----------------|
|                 | <i>nonconst</i> | <i>c</i>        | <i>d</i> ( $\neq c$ ) | <i>undef</i>    |
| <i>nonconst</i> | <i>nonconst</i> | <i>nonconst</i> | <i>nonconst</i>       | <i>nonconst</i> |
| <i>c</i>        | <i>nonconst</i> | <i>c</i>        | <i>nonconst</i>       | <i>c</i>        |
| <i>undef</i>    | <i>nonconst</i> | <i>c</i>        | <i>d</i>              | <i>undef</i>    |

Fig. 10.58.  $\rho(x)$  in terms of  $\mu(x)$  and  $v(x)$ .

1. The identity function is in  $F$ ; this function reflects any block that has no definition statements. If  $I$  is the identity function, and  $\mu$  is any mapping from variables to values, then  $I(\mu) = \mu$ . Note that  $\mu$  itself need not be the identity; it is arbitrary.
2. For each variable  $x$  and constant  $c$  there is a function  $f$  in  $F$  such that for every mapping  $\mu$  in  $V$ , we have  $f(\mu) = v$ , where  $v(w) = \mu(w)$  for all  $w$  other than  $x$ , and  $v(x) = c$ . These functions reflect the action of an assignment statement  $x := c$ .
3. For each three (not necessarily distinct) variables  $x$ ,  $y$ , and  $z$ , there is a function  $f$  in  $F$  such that for every mapping  $\mu$  in  $V$ , we have  $f(\mu) = v$ . The mapping  $v$  is defined by: for every  $w$  besides  $x$  we have  $v(w) = \mu(w)$ , and  $v(x) = \mu(y) + \mu(z)$ . If either  $\mu(y)$  or  $\mu(z)$  is *nonconst*, then the sum is *nonconst*. If either  $\mu(y)$  or  $\mu(z)$  is *undef*, but neither is *nonconst*, then the result is *undef*. This function expresses the effect of the assignment  $x := y + z$ . As usual in this chapter,  $+$  may be thought of as a generic operator; here an obvious modification is necessary if the operator is unary, ternary, or higher, and another obvious modification is needed to take into account effect of a copy statement,  $x := y$ .
4. For each variable  $x$  there is a function  $f$  in  $F$  such that for each  $\mu$ ,  $f(\mu) = v$ , where  $v(w) = \mu(w)$  for all  $w$  other than  $x$ , and  $v(x) = \text{nonconst}$ . This function reflects a definition by reading  $x$ , since after a read statement,  $x$  surely must be assumed not to have any particular constant value.  $\square$

### The Axioms of Data-Flow Analysis Frameworks

In order to make the kinds of data-flow algorithms that we have seen so far work for an arbitrary framework, we need to assume some things about the set  $V$ , the set of functions  $F$ , and the meet operator  $\wedge$ . Our basic assumptions are listed below, although some data-flow algorithms need additional assumptions.

1.  $F$  has an identity function  $I$ , such that  $I(\mu) = \mu$  for all  $\mu$  in  $V$ .
2.  $F$  is closed under composition; that is, for any two functions  $f$  and  $g$  in  $F$ , the function  $h$  defined by  $h(\mu) = g(f(\mu))$  is in  $F$ .
3.  $\wedge$  is an associative, commutative, and idempotent operation. These three properties are expressed algebraically as

$$\mu \wedge (\nu \wedge \rho) = (\mu \wedge \nu) \wedge \rho$$

$$\mu \wedge \nu = \nu \wedge \mu$$

$$\mu \wedge \mu = \mu$$

for all  $\mu, \nu$ , and  $\rho$  in  $V$ .

4. There is a *top element*  $\top$  in  $V$ , satisfying the law

$$\top \wedge \mu = \mu$$

for all  $\mu$  in  $V$ .

**Example 10.43.** Let us consider reaching definitions.  $F$  surely has the identity, the function where *gen* and *kill* are both the empty set. To show closure under composition, suppose we have two functions

$$f_1(X) = G_1 \cup (X - K_1)$$

$$f_2(X) = G_2 \cup (X - K_2)$$

Then

$$f_2(f_1(X)) = G_2 \cup ((G_1 \cup (X - K_1)) - K_2)$$

We may verify that the right side of the above is algebraically equal to

$$(G_2 \cup (G_1 - K_2)) \cup (X - (K_1 \cup K_2))$$

If we let  $K = K_1 \cup K_2$  and  $G = (G_2 \cup (G_1 - K_2))$ , then we have shown that the composition of  $f_1$  and  $f_2$ , which is  $f(X) = G \cup (X - K)$ , is of the form that makes it a member of  $F$ .

As for the meet operator, which is union, it is easy to check that union is associative, commutative, and idempotent. The "top" element turns out to be the empty set in this case, since  $\phi \cup X = X$  for any set  $X$ .

When we consider available expressions, we find that the same arguments used for reaching definitions also show that  $F$  has an identity and is closed under composition. The meet operator is now intersection, but this operator too is associative, commutative, and idempotent. The top element makes more intuitive sense this time; it is the set  $E$  of all expressions in the program, since for any set  $X$  of expressions,  $E \cap X = X$ .  $\square$

**Example 10.44.** Let us consider the constant computation framework introduced in Example 10.42. The set of functions  $F$  was designed to include the identity and to be closed under composition. To check the algebraic laws for

$\wedge$ , it suffices to show that they apply for each variable  $x$ . As an instance, we shall check idempotence. Let  $\nu = \mu \wedge \mu$ , that is, for all  $x$ ,  $\nu(x) = \mu(x) \wedge \mu(x)$ . It is simple to check by cases that  $\nu(x) = \mu(x)$ . For example, if  $\mu(x) = \text{nonconst}$ , then  $\nu(x) = \text{nonconst}$ , since the result of pairing *nonconst* with itself in Fig. 10.58 is *nonconst*.

Finally, the top element is the mapping  $\tau$  defined by  $\tau(x) = \text{undef}$  for all variables  $x$ . We can check from Fig. 10.58 that for any mapping  $\mu$  and any variable  $x$ , if  $\nu$  is the function  $\tau \wedge \mu$ , then  $\nu(x) = \mu(x)$ , since the result of pairing *undef* with any value in Fig. 10.58 is that other value.  $\square$

### Monotonicity and Distributivity

We need another condition to make the iterative algorithm for data-flow analysis work. This condition, called monotonicity, says informally that if you take any function  $f$  from the set  $F$ , and if you apply  $f$  to two members of  $V$ , one “bigger” than the other, then the result of applying  $f$  to the bigger is not less than what you get by applying  $f$  to the smaller.

To make the notion of “bigger” precise, we define a relation  $\leq$  on  $V$  by

$$\mu \leq \nu \text{ if and only if } \mu \wedge \nu = \mu$$

**Example 10.45.** In the reaching definition framework, where meet is union and members of  $V$  are sets of definitions,  $X \leq Y$  means  $X \cup Y = X$ , that is,  $X$  is a superset of  $Y$ . Thus,  $\leq$  looks “backward;” smaller elements of  $V$  are supersets of larger ones.

For available expressions, where meet is intersection, things work “right,” and  $X \leq Y$  means that  $X \cap Y = X$ ; i.e.,  $X$  is a subset of  $Y$ .  $\square$

Note from Example 10.45 that  $\leq$  in our sense need not have all the properties of  $\leq$  on integers. It is true that  $\leq$  is transitive; the reader may prove, as an exercise in using the axioms for  $\wedge$ , that  $\mu \leq \nu$  and  $\nu \leq \rho$  imply  $\mu \leq \rho$ . However,  $\leq$  in our sense is not a total order. For example, in the available expressions framework, we can have two sets  $X$  and  $Y$ , neither of which is a subset of the other, in which case neither  $X \leq Y$  nor  $Y \leq X$  would be true.

It often helps to draw the set  $V$  in a *lattice diagram*, which is a graph whose nodes are the elements of  $V$ , and whose edges are directed downward, from  $X$  to  $Y$  if  $Y \leq X$ . For example, Fig. 10.59 shows the set  $V$  for a reaching definitions data-flow problem where there are three definitions,  $d_1$ ,  $d_2$ , and  $d_3$ . Since  $\leq$  is “superset of,” an edge is directed downward from any subset of these three definitions to each of its supersets. Since  $\leq$  is transitive, we conventionally omit the edge from  $X$  to  $Y$  if there is another path from  $X$  to  $Y$  left in the diagram. Thus, although  $\{d_1, d_2, d_3\} \leq \{d_1\}$ , we do not draw this edge since it is represented by the path through  $\{d_1, d_2\}$ , for example.

It is also useful to note that we can read the meet off such diagrams, since  $X \wedge Y$  is always the highest  $Z$  for which there are paths downward to  $Z$  from both  $X$  and  $Y$ . For example, if  $X$  is  $\{d_1\}$  and  $Y$  is  $\{d_2\}$ , then  $Z$  in Fig. 10.59 is  $\{d_1, d_2\}$ , which makes sense, because the meet operator is union. It is also

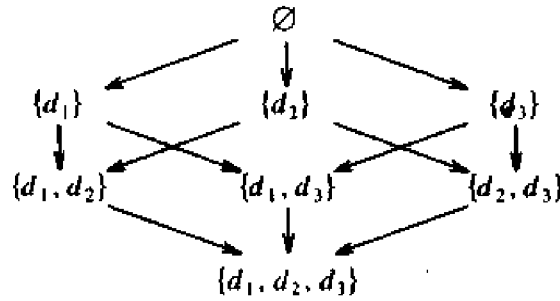


Fig. 10.59. Lattice of subsets of definitions.

true that the top element will appear at the top of the lattice diagram; that is, there is a path downward from  $T$  to each element.

Now, we can define a framework  $(F, V, \wedge)$  to be *monotone* if

$$\mu \leq \nu \text{ implies } f(\mu) \leq f(\nu) \quad (10.15)$$

for all  $\mu$  and  $\nu$  in  $V$ , and  $f$  in  $F$ .

There is an equivalent way to define monotonicity:

$$f(\mu \wedge \nu) \leq f(\mu) \wedge f(\nu) \quad (10.16)$$

for all  $\mu$  and  $\nu$  in  $V$  and  $f$  in  $F$ . It is useful to jump back and forth between these two equivalent definitions, so we shall sketch a proof of their equivalence, leaving for the reader some simple observations to verify, using the definition of  $\leq$  and the associative, commutative, and idempotent laws for  $\wedge$ .

Let us assume (10.15) and show why (10.16) holds. First, note that for any  $\mu$  and  $\nu$ ,  $\mu \wedge \nu \leq \mu$  and  $\mu \wedge \nu \leq \nu$  both hold; it is a simple proof, left for the reader, to prove these facts by proving, for any  $x$  and  $y$ , that  $(x \wedge y) \wedge y = x \wedge y$ . Thus, by (10.15),  $f(\mu \wedge \nu) \leq f(\mu)$  and  $f(\mu \wedge \nu) \leq f(\nu)$ . We leave it to the reader to verify the general law that

$$x \leq y \text{ and } x \leq z \text{ imply } x \leq y \wedge z$$

Letting  $x$  be  $f(\mu \wedge \nu)$ ,  $y = f(\mu)$ , and  $z = f(\nu)$ , we have (10.16).

Conversely, let us assume (10.16) and prove (10.15). We suppose  $\mu \leq \nu$  and use (10.16) to conclude  $f(\mu) \leq f(\nu)$ , thus proving (10.15). Equation (10.16) tells us  $f(\mu \wedge \nu) \leq f(\mu) \wedge f(\nu)$ . But since  $\mu \leq \nu$  is assumed,  $\mu \wedge \nu = \mu$ , by definition. Thus (10.16) says  $f(\mu) \leq f(\mu) \wedge f(\nu)$ . As a general rule the reader may show that

$$\text{if } x \leq y \wedge z \text{ then } x \leq z$$

Thus, (10.16) implies  $f(\mu) \leq f(\nu)$ , and we have proved (10.15).

Often, a framework obeys a condition stronger than (10.16), which we call the *distributivity condition*:

$$f(\mu \wedge \nu) = f(\mu) \wedge f(\nu)$$

for all  $\mu$  and  $\nu$  in  $V$  and  $f$  in  $F$ . Certainly, if  $x = y$ , then  $x \wedge y = x$  by idempotence, so  $x \leq y$ . Thus, distributivity implies monotonicity.

**Example 10.46.** Consider the reaching definitions framework. Let  $X$  and  $Y$  be sets of definitions, and let  $f$  be a function defined by  $f(Z) = G \cup (Z - K)$  for some sets of definitions  $G$  and  $K$ . Then we can verify that the reaching definitions framework satisfies the distributivity condition, by checking that

$$G \cup ((X \cup Y) - K) = (G \cup (X - K)) \cup (G \cup (Y - K))$$

Drawing a Venn diagram makes the proof of the above relationship transparent, although it looks complicated.  $\square$

**Example 10.47.** Let us show that the constant computation framework is monotone, but not distributive. First, it helps to apply the  $\wedge$  operation and the  $\leq$  relationship to the elements appearing in the table of Fig. 10.58. That is, let us define

$$\begin{aligned} nonconst \wedge c &= nonconst && \text{for any constant } c \\ c \wedge d &= nonconst && \text{for constants } c \neq d \\ c \wedge undef &= c && \text{for any constant } c \\ nonconst \wedge undef &= nonconst \\ x \wedge x &= x && \text{for any value } x \end{aligned}$$

Then Fig. 10.58 can be interpreted as saying that  $\rho(a) = \mu(a) \wedge \nu(a)$ .

We can determine what the  $\leq$  relationship on values is from the  $\wedge$  operation. We find

$$\begin{aligned} nonconst &\leq c && \text{for any constant } c \\ c &\leq undef && \text{for any constant } c \\ nonconst &\leq undef \end{aligned}$$

This relationship is shown in the lattice diagram of Fig. 10.60, where the  $c_i$ 's are intended to suggest all possible constants. Note that that figure is not  $\leq$  on elements of  $V$ ; rather it is a relation on the set of values for  $\mu(a)$  for individual variables  $a$ . The elements of  $V$  may be thought of as vectors of such values, one component for each variable, and the lattice diagram for  $V$  can be extrapolated from Fig. 10.60 if we remember that  $\mu \leq \nu$  holds if and only if  $\mu(a) \leq \nu(a)$  for all  $a$ ; i.e., if the vectors representing  $\mu$  and  $\nu$  have each component related by  $\leq$ , and the relationship is in the same direction in each component.

Thus, to say  $\mu \leq \nu$  is to say that whenever  $\mu(a)$  is a constant  $c$ ,  $\nu(a)$  is either that constant or *undef*, and whenever  $\mu(a)$  is *undef*, so is  $\nu(a)$ . A careful examination of the various functions  $f$  that are associated with the different kinds of definition statements enables us to verify that if  $\mu \leq \nu$ , then  $f(\mu) \leq f(\nu)$ , thus proving (10.15) and showing monotonicity. For example, if  $f$  is associated with assignment  $a := b + c$ , only  $\mu(a)$  and  $\nu(a)$  change, so we have to check that if  $\mu \leq \nu$  — i.e.,  $\mu(x) \leq \nu(x)$ , for all  $x$  — then

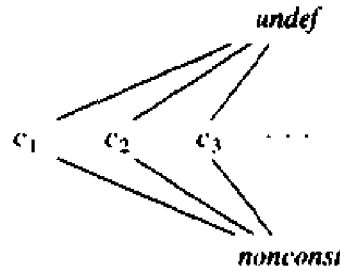


Fig. 10.60. Lattice diagram for values of variables.

$[f(\mu)](a) \leq [f(v)](a)$ .<sup>14</sup> We must consider all possible values of  $\mu(b)$ ,  $\mu(c)$ ,  $v(b)$ , and  $v(c)$ , subject to the constraints that  $\mu(b) \leq v(b)$  and  $\mu(c) \leq v(c)$ . For example, if

$$\begin{aligned}\mu(b) &= \text{nonconst} \\ v(b) &= 2 \\ \mu(c) &= 3 \\ v(c) &= \text{undef}\end{aligned}$$

then  $[f(\mu)](a) = \text{nonconst}$  and  $[f(v)](a) = \text{undef}$ . Since  $\text{nonconst} \leq \text{undef}$ , we have made the check in one case. The other cases are left as an exercise for the reader.

Now, we must check our claim that the constant computation framework is not distributive. For this part, let  $f$  be the function associated with the assignment  $a := b + c$ , and let  $\mu(b) = 2$ ,  $\mu(c) = 3$ ,  $v(b) = 3$ , and  $v(c) = 2$ . Let  $\rho = \mu \wedge v$ . Then  $\mu(b) \wedge v(b) = 2 \wedge 3 = \text{nonconst}$ . Similarly,  $\mu(c) \wedge v(c) = \text{nonconst}$ . Equivalently,  $\rho(b) = \rho(c) = \text{nonconst}$ . It follows that  $[f(\rho)](a) = \text{nonconst}$ , since the sum of two nonconstant values is presumed to be nonconstant.

On the other hand,  $[f(\mu)](a) = 5$ , since given  $b = 2$  and  $c = 3$ , the assignment  $a := b + c$  sets  $a$  to 5. Similarly,  $[f(v)](a) = 5$ . Thus,  $[f(\mu) \wedge f(v)](a) = 5$ . We now see that  $\rho(a) = [f(\mu \wedge v)](a) \neq [f(\mu) \wedge f(v)](a)$ , so the distributivity condition is violated.

Intuitively, the reason we get a distributivity violation is that the constant computation framework is not powerful enough to remember all invariants, in particular, the fact that along paths whose effects on variables are described by either  $\mu$  or  $v$ , the equation  $b + c = 5$  holds, even though neither  $b$  nor  $c$  are themselves constant. We could devise more complicated frameworks to avoid this particular problem, although the payoff from doing so is not clear. Fortunately, monotonicity is adequate for the iterative data-flow algorithm to “work,” as we shall next see.  $\square$

<sup>14</sup> We have to be careful reading an expression like  $[f(\mu)](a)$ . It says, we apply  $f$  to  $\mu$  to get some mapping  $f(\mu)$ , which we call  $\mu'$ . Then, we apply  $\mu'$  to  $a$ , and the result is one of the values in the diagram of Fig. 10.60.

### Meet-Over-Paths Solutions to Data-Flow Problems

Let us imagine that a flow graph has associated with each of its nodes a transfer function, one of the functions in the set  $F$ . For each block  $B$ , let  $f_B$  be the transfer function for  $B$ .

Consider any path  $P = B_0 \rightarrow B_1 \rightarrow \cdots \rightarrow B_k$  from the initial node  $B_0$  to some block  $B_k$ . We can define the *transfer function for  $P$*  to be the composition of  $f_{B_0}, f_{B_1}, \dots, f_{B_{k-1}}$ . Note that  $f_{B_k}$  is not part of the composition, reflecting the point of view that this path is taken to reach the beginning of block  $B_k$ , not its end.

We have assumed that the values in  $V$  represent information about data used by the program, and that the confluence operator  $\wedge$  tells how that information is combined when paths converge. It also makes sense to see the top element as representing "no information," since a path carrying the top element yields to any other path, as far as what information is carried after confluence of the paths. Thus, if  $B$  is a block in the flow graph, the information entering  $B$  should be computable by considering every possible path from the initial node to  $B$  and seeing what happens along that path, starting out with no information. That is, for each path  $P$  from  $B_0$  to  $B$ , we compute  $f_P(T)$  and take the meet of all the resulting elements.

In principle, this meet could be over an infinite number of different values, since there are an infinite number of different paths. In practice, it is often adequate to consider only acyclic paths, and even when it is not, as for the constant computation framework discussed above, there are usually other reasons we can find to make this infinite meet be finite for any particular flow graph.

Formally, we define the *meet-over-paths solution* to a flow graph to be

$$mop(B) = \bigwedge_{\substack{\text{paths } P \\ \text{from } B_0 \text{ to } B}} f_P(T)$$

The *mop* solution to a flow graph makes sense when we realize that as far as information reaching block  $B$  is concerned, the flow graph may as well be the one suggested in Fig. 10.61, where the transfer function associated with each of the (possibly infinite number of) paths  $P_1, P_2, \dots$ , in the original flow graph has been given an entirely separate path to  $B$ . In Fig. 10.61, the information reaching  $B$  is given by the meet over all paths.

### Conservative Solutions to Flow Problems

When we try to solve the data-flow equations that come from an arbitrary framework, we may or may not be able to obtain the *mop* solution easily. Fortunately, as with the concrete examples of data-flow frameworks in Sections 10.5 and 10.6, there is a safe direction in which to err, and the iterative data-flow algorithm we discussed in those sections turns out always to provide us with a safe solution. We say a solution  $in[B]$  is a *safe solution* if  $in[B] \leq mop(B)$  for all blocks  $B$ .

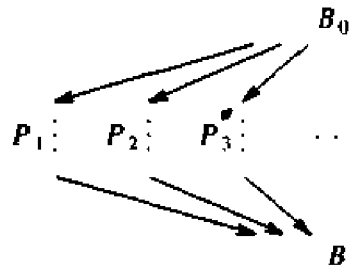


Fig. 10.61. Graph showing the set of all possible paths to  $B$ .

Despite what the reader might imagine, we did not pull this definition out of thin air. Recall that in any flow graph, the set of *apparent* paths to a node (those that are paths in the flow graph) can be a proper subset of the *real* paths, those that are taken on some execution of the program corresponding to this flow graph. In order that the result of the data-flow analysis be usable for whatever purpose we intend it, the data must still be safe if we modify the flow graph by deleting some paths, since we cannot in general distinguish real paths from apparent paths that are not real.

Suppose among the infinite set of paths suggested in Fig. 10.61,  $x$  is the meet of  $f_P(T)$  taken over all real paths  $P$  that are followed on some execution. Also, let  $y$  be the meet of  $f_P(T)$  over all other paths  $P$ . Thus,  $\text{mop}(B)$  is  $x \wedge y$ . Then the true answer to our data-flow problem at node  $B$  is  $x$ , but the *mop* solution is  $x \wedge y$ . Recall that  $x \wedge y \leq y$ , since  $(x \wedge y) \wedge y = x \wedge y$ . Thus, the *mop* solution is  $\leq$  the true solution.

While we may prefer the “true” solution to the data-flow problem, we shall almost surely have no efficient way to tell exactly which paths are real and which are not, so we are forced to accept the *mop* solution as the closest feasible solution. Thus, whatever use we make of the data-flow information must be consistent with the possibility that the solution we obtain is  $\leq$  the true solution. Once we accept that, we should also be able to accept a solution that is  $\leq$  the *mop* (and therefore  $\leq$  the true solution). Such solutions are easier to obtain than the *mop* for those frameworks that are monotone but not distributive. For distributive frameworks, like those of Section 10.6, the simple iterative algorithm computes the *mop* solution.

### The Iterative Algorithm for General Frameworks

There is an obvious generalization of Algorithm 10.2 that works for a large variety of frameworks. The iterative algorithm requires that the framework be monotone, and it requires finiteness, in the sense that the meet over the infinite set of paths suggested in Fig. 10.61 is equivalent to a meet over a finite subset. We shall give the algorithm and then discuss the ways in which we could assure finiteness. However, one common guarantee of finiteness is the one we have had all along: propagation along acyclic paths is sufficient.



**Algorithm 10.18.** Iterative solution to general data-flow frameworks.

*Input.* A data-flow graph, a set of “values”  $V$ , a set of functions  $F$ , a meet operation  $\wedge$ , and an assignment of a member of  $F$  to each node of the flow graph.

*Output.* A value  $in[B]$  in  $V$  for each node of the flow graph.

*Method.* The algorithm is given in Fig. 10.62. As with the familiar iterative data-flow algorithms, we compute  $in$  and  $out$  for each node by successive approximations. We assume that  $f_B$  is the function in  $F$  associated with block  $B$ ; that function plays the role of *gen* and *kill* in Section 10.6.  $\square$

```

1)  for each node  $B$  do /* initialize, assuming  $in[B] = \top$  */
2)       $out[B] := f_B[\top]$ ;
3)  while changes to any  $out$  occur do
4)      for each block  $B$ , in depth-first order do begin
5)           $in[B] := \bigwedge_{\substack{\text{predecessors} \\ P \text{ of } B}} out[P]$ ;
          /* in above, the meet of an empty set is  $\top$  */
6)           $out[B] := f_B(in[B])$ 
      end

```

Fig. 10.62. Iterative algorithm for general frameworks.

### A Data-Flow Analysis Tool

We can now see the way the ideas of this section can be applied to a tool for data-flow analysis. Algorithm 10.18 depends for its working on the following subroutines.

1. A routine to apply a given  $f_B$  in  $F$  to a given value in  $V$ . This routine is used in lines (2) and (6) of Fig. 10.62.
2. A routine to apply the meet operator to two values in  $V$ ; this routine is needed zero or more times in line (5).
3. A routine to tell whether two values are equal. This test is not made explicitly in Fig. 10.62, but it is implicit in the test for change in any of the values of  $out$ .

We also need to have specific data type declarations for  $F$  and  $V$ , so we can pass arguments to the routines mentioned above. The values of  $in$  and  $out$  in Fig. 10.62 are also of the type declared for  $V$ . Finally, we need a routine that will take the ordinary representation of the contents of a basic block, that is, a list of statements, and produce an element of  $F$ , the transfer function for that block.

**Example 10.48.** For the reaching definitions framework, we might first build a table that identified each statement of the given flow graph with a unique integer from 1 up to some maximum  $m$ . Then, the type of  $V$  could be bit vectors of length  $m$ .  $F$  could be represented by pairs of bit vectors of that size, i.e., by the *gen* and *kill* sets. The routine to construct the *gen* and *kill* bit vectors, given the statements of a block and the table associating definition statements with bit-vector positions, is straightforward, as are the routines to compute meets (logical or of bit vectors), compare bit vectors for equality, and apply functions defined by a *gen-kill* pair to bit vectors.  $\square$

The data-flow analysis tool is thus little more than an implementation of Fig. 10.62 with calls to the given subroutines whenever a meet, function application, or comparison is needed. The tool would support a fixed representation of flow graphs, and thus be able to perform tasks like finding all the predecessors of a node, finding the depth-first ordering of the flow graph, or applying to each block the routine that computes the function in  $F$  associated with that block. The advantage of using such a tool is that the graph-manipulation and convergence-checking aspects of Algorithm 10.18 do not have to be rewritten for each data-flow analysis that we do.

### Properties of Algorithm 10.18

We should make clear the assumptions under which Algorithm 10.18 actually works and exactly what the algorithm converges to when it does converge. First, if the framework is monotone and converges, then we claim the result of the algorithm is that  $in[B] \leq mop(B)$  for all blocks  $B$ . The intuitive reason is that along any path  $P = B_0, B_1, \dots, B_k$  from the initial node to  $B = B_k$ , we can show by induction on  $i$  that the effect of the path from  $B_0$  to  $B_i$  is felt after at most  $i$  iterations of the while-loop in Fig. 10.62. That is, if  $P_i$  is the path  $B_0, \dots, B_i$ , then after  $i$  rounds,  $in[B_i] \leq f_{P_i}(T)$ . Thus, when and if the algorithm converges,  $in[B]$  will be  $\leq f_P(T)$  for every path  $P$  from  $B_0$  to  $B$ . Using the rule that if  $x \leq y$  and  $x \leq z$ , then  $x \leq y \wedge z$ ,<sup>15</sup> we can show that  $in[B] \leq mop(B)$ .

When the framework is distributive, we can show Algorithm 10.18 does in fact converge to the *mop* solution. The essential idea is to prove that at all times during the running of the algorithm,  $in[B]$  and  $out[B]$  are each equal to the meet of  $f_P(T)$  for some set of paths  $P$  to the beginning and end of  $B$ , respectively. However, we show in the next example that need not be the case when the framework is monotone but not distributive.

**Example 10.49.** Let us exploit the example of nondistributivity of the

<sup>15</sup> There is the technicality that we must in principle show this rule not just for two values,  $y$  and  $z$  (from which follows the rule that if  $x \leq y_i$  for any finite set of  $y_i$ 's then  $x \leq \bigwedge y_i$ ) but that the same rule holds for an infinite number of  $y_i$ 's. However, in practice, whenever we get convergence of Algorithm 10.18, we shall find a finite number of paths such that the meet over all paths is equal to the meet over this finite set.